EIE330 Report from Group 8

Deng Yingying (1230002196)
Yang Kaitian (1220010941)
Yu Kelin (1230020934)

Instructor: Professor Wang Ze

December 23, 2025

Contents

Chapter 1

Breakout Video Game

Code Repository

Final project (submission): GitHub: Final_Code_Submission
Development history (full repo): GitHub: fpga-final-project

Demonstration Videos

Path (1): START → GAME OVER

- Video link: `https://www.bilibili.com/video/BV1JWB4BKEpe`

Path (2): START → CLEAR

(i) Original-speed video link: `https://www.bilibili.com/video/BV1VnBTBqERr`

(ii) 4×-speed video link: `https://www.bilibili.com/video/BV1VJBTBTEg2`

1.1 Introduction

Background

Brick-breaker (Breakout) is a classical 2D arcade-style game that integrates real-time rendering, user interaction, and rigid-body-like collision behaviors. Implementing such a game on FPGA is a representative course-project setting, because it requires a complete hardware-oriented workflow: pixel-level VGA rendering, frame-synchronous state updates, synchronous input handling, and modular design under timing constraints. Compared with a software implementation, an FPGA implementation must explicitly design the game loop and the interaction between display

timing (pixel scanning) and game logic (object motion, collision response, and state machines).
A key technical challenge is reliable collision detection under discrete-time updates. If the ball moves several pixels per frame, naive "check position at the next frame" logic can miss the true collision moment (tunneling), especially for thin objects such as bricks or the paddle. Therefore, a robust design typically needs a continuous-time or swept collision view within each frame, and then converts the result into a discrete state update. This report presents our design and verification of a brick-breaker game on FPGA, with an emphasis on practical and robust collision handling and a clear finite-state-machine (FSM) structure for the overall game process.

Purpose

The purpose of this course project is to design and implement a complete brick-breaker video game on an FPGA-based VGA platform, and to demonstrate a structured hardware design methodology.
Specifically, our objectives are:

Ⓐ Implement a real-time VGA display system that can render multiple pages and game objects (ball, paddle, bricks, UI elements) in a stable and visually consistent manner.

Ⓑ Develop a reliable collision detection and response mechanism for ball–wall, ball–paddle, and ball–brick interactions, including the multiple-brick case.

Ⓒ Introduce an FSM-based game process controller, including a START page, normal gameplay, life management (hearts), and terminal pages (CLEAR / GAME OVER) with correct transitions.

Ⓓ Verify the correctness of each key module and the end-to-end game process using simulation (virtual development board based) and recorded videos.

Key Results

We achieved a complete FPGA-based brick-breaker game with the following key results:

Ⓐ **Modular VGA rendering pipeline.** Each page/object is implemented as a dedicated VGA picture module, and the top-level composition selects pixel data according to the current game state and pixel coordinates.

Ⓑ **Robust collision handling.** For wall collisions, we clamp the ball position to the valid boundary and reverse the corresponding velocity component. For brick collisions, we adopt a "single-brick to multiple-brick" strategy: within one frame, we compute candidate collision intervals and determine the earliest valid collision to avoid missing collisions in the multi-brick setting.

Ⓒ **Clear game-process logic via FSM.** A main FSM manages page-level states (START / GAME / CLEAR / GAME OVER), and a sub-FSM manages gameplay events such as life reduction, reset behavior, and win/loss detection, ensuring deterministic transitions.

Ⓓ **End-to-end verification.** We validated two representative paths: (1) START → GAME OVER and (2) START → CLEAR, and provided accelerated and/or original-speed videos to demonstrate correctness under realistic interaction.

1.1.1 Contributions

Yang Kaitian (1220010941)

Note: Below, "he/his/him" refers to Yang Kaitian. In the AI-usage statement, "it" refers to the AI tool.

- **Project Design, Implementation, Simulation, and Test**
  - **Design**
    - Proposed the overall system design with three core tasks: *(i) game process*, *(ii) page controllers*, and *(iii) VGA rendering / composition*.
    - Designed the game-process logic (excluding health points and the ball color-change feature), including:
      - ball–wall collision and reflection logic;
      - ball–single-brick collision and reflection logic;
      - ball–multiple-bricks collision and reflection logic;
      - ball–paddle (board) collision and reflection logic;
      - button debounce logic;
      - button-controlled paddle movement with wall constraints.
    - Designed the page-controller architecture, including the *Main FSM* and *Sub FSM*.
  - **Implementation**

    - Implemented all corresponding Verilog modules for the above designs.
    - Refactored and reconstructed some teammate-provided code. For example, a mixed rendering module was split into independent modules: `vga_pic_gameover.v`, `vga_pic_start.v`, `vga_pic_clear.v`, and `hearts_overlay.v`.
  - **Simulation and Verification**
    - Performed virtual development board based simulations for all major modules above.
    - Built waveform-based simulations (testbenches) for the Main FSM and Sub FSM to verify state transitions and key signals.
- **Report Writing**
  - Wrote the following major parts of the report:
    - "Prepare FPGA Solution for Game Process"
    - "Prepare FPGA Solution for Page Controllers"
    - "Implementations for Game Process"
    - "Implementations for Page Controllers"
    - "Implementations for Overall Combination"
    - "Virtual Development Board Based Simulation for Game Process" (Functional Simulation in Results)
    - "Functional Simulation for Page Controllers" (Functional Simulation in Results)
    - "Test"
    - "Discussion"
- **Presentation**
  - Served as the sole presenter representing the group.
  - Prepared slides **1–27** of the presentation deck.

Generative AI Usage Statement (ChatGPT, Gemini)

- **Project Design / Implementation / Simulation**

  - **Design:** ChatGPT suggested considering the swept-AABB idea for ball–brick collision. Other design decisions were either developed independently by Yang Kaitian or refined through AI-assisted discussion based on his own design thoughts.
  - **Implementation:** A typical workflow was: Yang Kaitian first wrote a small/simple Verilog draft to express the implementation idea, then used ChatGPT to expand it into a complete and reliable module. Some advanced parts (e.g., multiple-brick management logic) were produced by ChatGPT with multiple iterative rounds of debugging and correction.
  - **Simulation:** Video-based functional simulations are independent of AI tools. For the Main FSM testbench, Yang Kaitian specified the verification requirements and used ChatGPT to generate the testbench code. For the Sub FSM testbench, Gemini was additionally used to help fix bugs.
  - **Test:** The final testing videos and demonstrations are independent of AI tools.
- **Report Writing**
  - A typical workflow was: Yang Kaitian first drafted a short LaTeX skeleton (key ideas + structure), then used ChatGPT to rewrite it into a complete, consistent, and well-formatted LaTeX section.
  - The "Background", "Purpose", "Key Results", and "Conclusion" sections were summarized by ChatGPT based on the other written parts of this report.

Yu Kelin (1230020934)

Deng Yingying (1230002196)

1.2 Materials and Methods

1.2.1 Preliminary Research

Target Functions

The system is designed with the following expected functions.

Inputs.

- **Two functional buttons** for controlling the board (e.g., move left , move right). and entering the game from START page
- **One reset button** for going back to START after the game ending.

Outputs. A dynamic and static hybrid VGA video with object interactions, including a moving ball, a controllable board, bricks, and three static pages (START, GAME OVER, CLEAR).

Game Features.

Ⓐ **Ball motion and reflection:** the ball moves automatically and is reflected by the walls, bricks, and the board.

Ⓑ **Brick removal:** once the ball collides with a brick, the brick will be removed from the screen.

Ⓒ **Win condition:** when all bricks are removed, the system enters the **CLEAR** page.

Ⓓ **Health points:** the player is given **three** health points initially. If the ball falls out of the bottom of the screen, the player loses one health point and the ball is reset to the initial position.

Ⓔ **Lose condition:** when the health points decrease to zero, the system enters the **GAME OVER** page.

Ⓕ **Restart:** after entering the **CLEAR** or **GAME OVER** page, the user can press the reset button to go back to START page.

Required Devices

Virtual Development Board, VGA simulator

Design Plan

Design Principle Whole target → Small Pieces → Large Combination.
Step1: Divide the Big Target into many small, easy tasks.
Step2: Define logic relations/interfaces and complete separately.
Step3: Combine them; if problems, go back to Step1 or 2.
Nestedly Use, for each small task, divide it furthermore.

Three Core Tasks for whole system

Ⓐ Game Process Core Task: How to achieve the breakout game logic?

Ⓑ Page Controllers Core Task: How to change one page to another one?

Ⓒ Rendering Core Task: including static and dynamic pages

1.2.2 Prepare FPGA Solution for Game Process

Overview

Ⓐ **Section 1: Static template of video** (easy, just a static frame).

Ⓑ **Section 2: Video without Functional Buttons**

- reflection effect w.r.t. walls
- add bricks

Ⓒ **Section 3: Video with Functional Buttons**

- board movement, no ball
- board reflection on ball

For each section, we complete it and simulate it separately. Finally, combine all, and get the Main Game Page.
After analysing the 3 sections, we found all interactions are among 4 types entities, Walls, Ball, Board, Bricks. In the following, we will exploit the notion to classify the specific tasks for each mentioned section under the Game Process Core task.

Section 1: Rendering Basic Static Frame

Designed Waveforms or Thoughts

Section 2.1 ball vs walls

Designed Waveforms or Thoughts

In this beginning dynamic module, we expect the ball will be reflected between the three walls, right, left and top. If the ball touches the bottom boundary, it will disappear.

In design, we used a two-step method:

Ⓐ An automatic moving ball reflected between four walls

Ⓑ Set the bottom wall with ball disappearing effect

A. An automatic moving ball reflected between four walls First, the ball is assigned a constant velocity and moves in a constant-speed straight-line motion. Second, once the *predicted next position* of the ball is outside of the screen (i.e., beyond a wall boundary), the ball is reflected according to the mirror reflection principle.

1. Discrete-time motion model. Let (x_t, y_t) denote the center coordinates of the ball at discrete time step t. The velocity components (v_x, v_y) are defined in units of *pixels per frame*, which implicitly assumes a unit time step $\Delta t = 1$ for each update. The position update rule is therefore

$$\begin{aligned} x_{\text{next}} &= x_t + v_x, \\ y_{\text{next}} &= y_t + v_y. \end{aligned} \tag{1.1}$$

Here, each update corresponds to one video frame, so v_x and v_y represent the displacement per frame rather than per second.

2. Reflected between Four Walls. Based on the motion model, we design the following flow chart for logic clarity.

In the following, we explain the flow chart in two blocks: (1) the detailed reflection rule, and (2) the mathematical determination rule for "out of the screen".

2.1 Boundary prediction and "push-to-wall" (clamping) strategy. In our implementation, we do not allow the ball center to move outside the valid visible region. Instead, we first compute the predicted next position $(x_{\text{next}}, y_{\text{next}})$, and then *clamp* it to the boundary (i.e., "push" it onto the wall) if it would cross a wall.

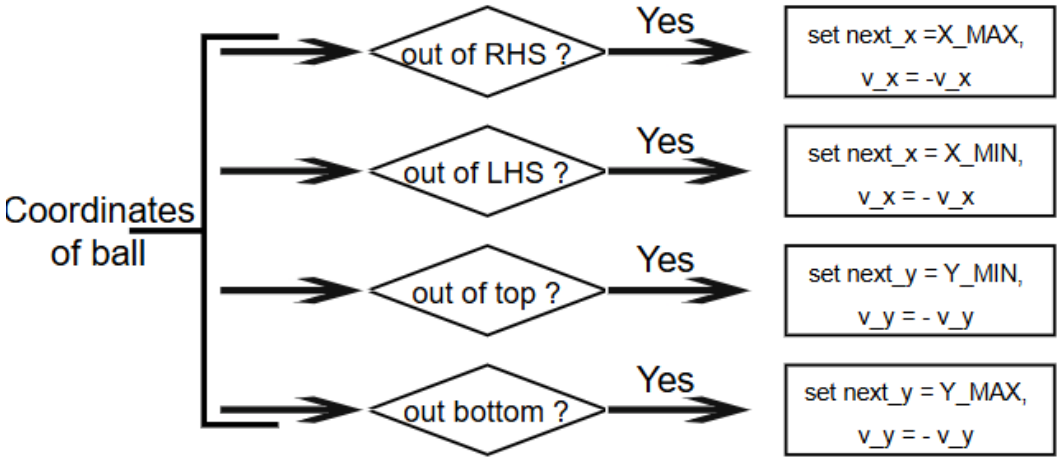

Coordinates of ball
out of RHS ?
Yes
set next_x =X_MAX, v_x = -v_x
out of LHS ?
Yes
set next_x = X_MIN, v_x = - v_x
out of top ?
Yes
set next_y = Y_MIN, v_y = - v_y
out bottom ?
Yes
set next_y = Y_MAX, v_y = - v_y

Fig. 1.1

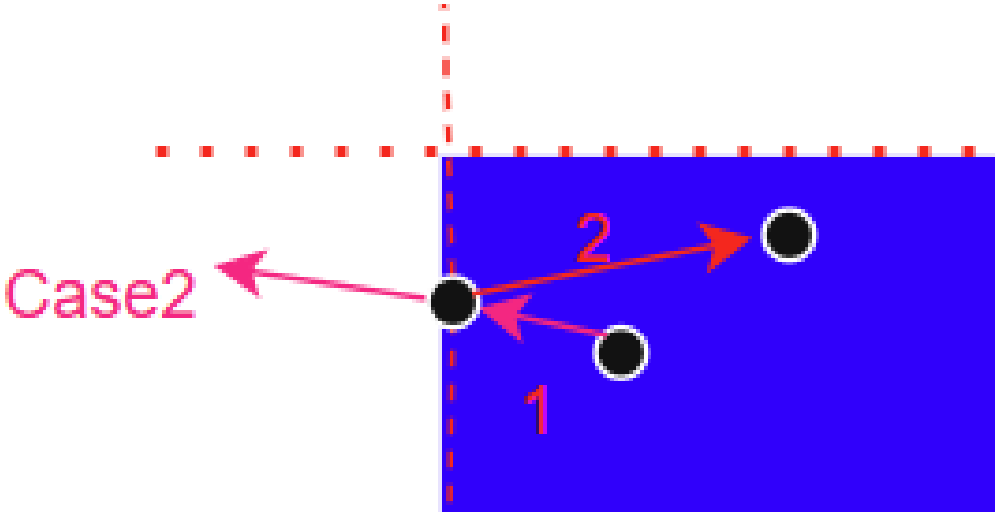

Case2
2
1

Fig. 1.2

Let the screen size be $W \times H$ (in pixels) and the ball radius be R. Then the allowable range of the ball center is

$$x \in [x_{\min}, x_{\max}] = [R,\, W - R], \qquad y \in [y_{\min}, y_{\max}] = [R,\, H - R]. \tag{1.2}$$

We update the ball position using the clipping function

$$x_{t+1} = \mathrm{clip}(x_{\text{next}}, x_{\min}, x_{\max}), \qquad y_{t+1} = \mathrm{clip}(y_{\text{next}}, y_{\min}, y_{\max}), \tag{1.3}$$

where

$$\mathrm{clip}(z, a, b) = \begin{cases} a, & z \le a, \\ b, & z \ge b, \\ z, & a < z < b. \end{cases} \tag{1.4}$$

Therefore, if the predicted position is beyond a wall, the ball center is written directly to the wall location ($x_{\min}$, $x_{\max}$, $y_{\min}$, or $y_{\max}$), which realizes the "push-to-wall" effect and prevents overshooting.

2.2 Reflection (velocity reversal) rule. After clamping the position, we reverse the corresponding velocity component according to the same boundary condition:

$$\text{if } \big(x_{\text{next}} \le x_{\min}\big) \text{ or } \big(x_{\text{next}} \ge x_{\max}\big), \quad v_x \leftarrow -v_x, \tag{1.5}$$

$$\text{if } \big(y_{\text{next}} \le y_{\min}\big) \text{ or } \big(y_{\text{next}} \ge y_{\max}\big), \quad v_y \leftarrow -v_y. \tag{1.6}$$

This implements an ideal mirror reflection on vertical/horizontal walls: the tangential component is unchanged, and the normal component is negated.

2.3 Mass-point modeling and boundary expansion ("shrink the ball to a point"). To simplify collision detection, we treat the ball (a disk of radius R) as a mass point located at its center. Equivalently, we expand the walls (or boundaries) by R using a Minkowski-sum interpretation: a collision between a disk and a wall is transformed into a collision between a point and an expanded wall. Thus, the motion constraint is enforced directly on the ball center.

Example: bottom boundary (deriving $y_{\max} = y_0 - R$). Assume the bottom wall (screen boundary) is located at $y = y_0$. Let the ball center at time t be y_t, and its predicted next center position be

$$y_{\text{next}} = y_t + v_y, \tag{1.7}$$

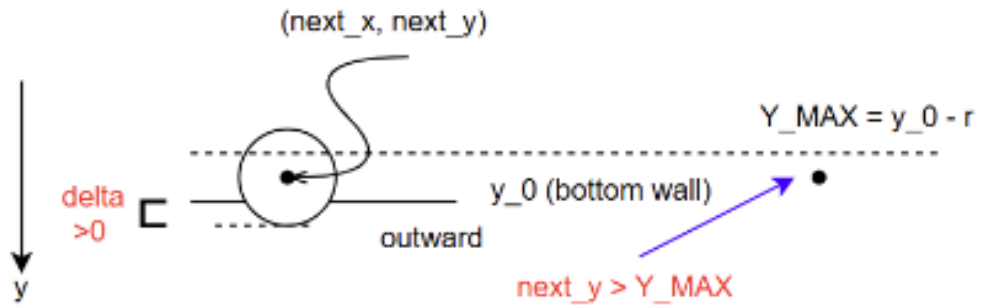


Fig. 1.3. Treat the ball as a point and expand the wall by radius R. A predicted step that makes $\delta > 0$ indicates the center crosses the expanded boundary.

where v_y is the per-frame displacement (pixels/frame). Since the ball has radius R, the lowest point of the ball after the update is $y_{\text{next}} + R$. Therefore, the ball would cross the bottom wall if

$$y_{\text{next}} + R > y_0. \tag{1.8}$$

Define the penetration distance (as in Fig. 1.3) by

$$\delta = (y_{\text{next}} + R) - y_0. \tag{1.9}$$

Then $\delta > 0$ indicates the ball is outside (penetrating the wall). Rearranging the inequality gives

$$y_{\text{next}} > y_0 - R. \tag{1.10}$$

Hence, the valid maximum center position is

$$y_{\max} = y_0 - R. \tag{1.11}$$

In implementation, this yields the boundary check

$$\text{if } y_{\text{next}} > y_{\max} \text{ then the ball would cross the bottom boundary.} \tag{1.12}$$

Generalization to four walls. Let the visible screen region be $x \in [0, W]$ and $y \in [0, H]$ (in pixels). After shrinking the ball to a point, the valid range of the ball center becomes

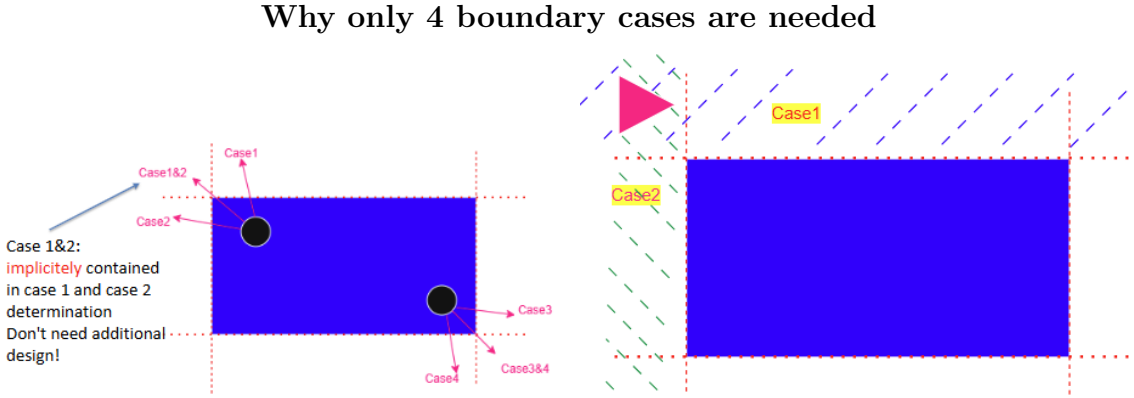
$$x \in [R,\ W - R], \qquad y \in [R,\ H - R], \tag{1.13}$$

i.e.,

$$x_{\min} = R, \quad x_{\max} = W - R, \quad y_{\min} = R, \quad y_{\max} = H - R. \tag{1.14}$$

Thus, out-of-screen prediction can be determined by comparing $(x_{\text{next}}, y_{\text{next}})$ with these expanded-boundary limits.

Only four basic boundary checks are needed (corner cases are implicit). Although the outside regions may look like "8 cases" if we partition the plane using the extended boundary lines, in logic design we only need four predicates (left, right, top, bottom). The corner cases are automatically handled as the conjunction of one horizontal and one vertical boundary condition (e.g., left+top, right+bottom), so no additional cases are required.



(a) Enumerating outside regions yields 8 visual regions, but corner cases are implicitly covered.

(b) Two half-plane masks (for x and y) overlap as a triangle, illustrating that only 4 basic wall checks are needed.

Fig. 1.4. Only four boundary checks are required: left wall, right wall, top wall, and bottom wall. The corner "8-region" cases are automatically handled as the simultaneous triggering of one horizontal and one vertical boundary condition (e.g., left+top).

B. Set the bottom wall with ball disappearing effect Based on the above design, the bottom disappearing effect is simple to design. Just removing the bottom wall clipping, the ball position will be calculated as a large number, such that the number is not inside the screen rendering range. In the following, we give the final flow chart for the ball and walls interaction design.

Note: Relation to the Implementation At the code level, the module for this section is named vga_pic_move2.v. But, in the final implementation, or so-called final breakout game code, it will be further combined with other seperately designed module for engineering clarity.

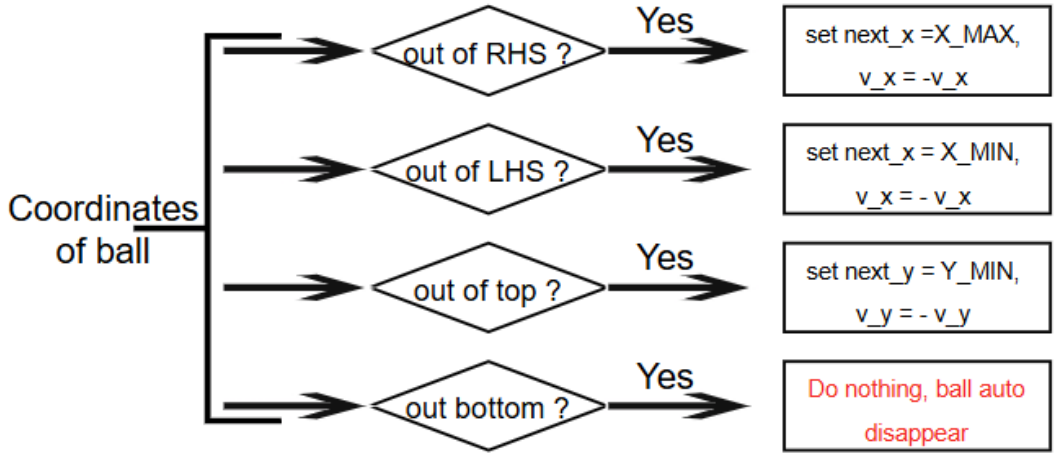


Fig. 1.5

Section 2.2.1 Single brick problem

Because the multiple-brick problem is complicated as an overall target, we first focus on the single brick problem.

We need to handle the interaction between the ball and one brick. For the brick side, the logic is simple: if the collision between the ball and the brick is detected, then the brick will be deleted (set `alive` to 0). For the ball side, if the collision is detected, then the ball needs to be reflected. As a result, we need to design a collision detection mechanism. The overall logic is summarized by the following flow chart.

Single brick logic flow

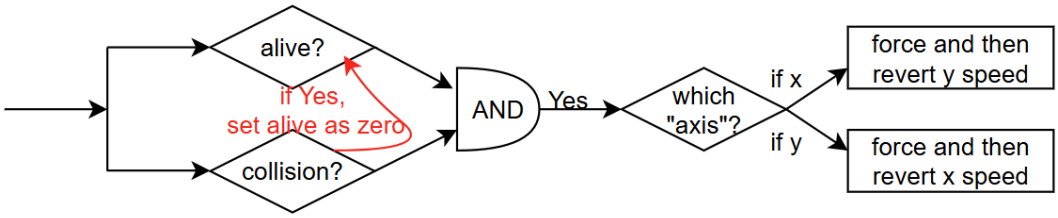


Fig. 1.6. Overall logic for brick deletion (`alive`) and ball reflection after collision detection.

From the flow chart, the detailed problems are: (1) how to determine whether there is a collision, and (2) how to know which axis the ball collides with. Besides them, we use an `alive` signal to denote whether the brick exists or not.

To answer the above two problems, we need to consider the specific situation be-

tween the ball and the brick. Fig. 1.7 shows two cases. In the left one, the ball is outside of the brick in the former frame, and is inside of the brick in the next frame. However, if the ball velocity is large enough, then as illustrated in the right-hand-side case, both positions of the ball in two neighbouring frames are outside of the brick, even though the ball actually passes through the brick during the middle process. Therefore, we cannot only check the ball position at each frame. We need a mathematical model to describe the continuous motion *within* one frame interval.

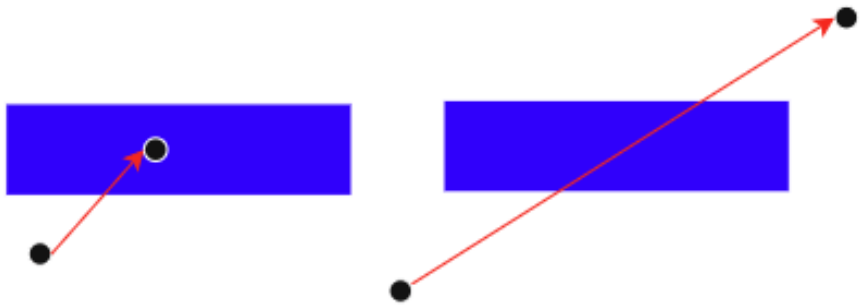


Fig. 1.7. Two neighbouring-frame positions may both be outside of the brick while the trajectory intersects the brick in between.

Mass point and orthogonal projection We use the mass point strategy again and apply orthogonal projection to transform the 2D collision problem in the VGA plane into two 1D problems (x-direction and y-direction), which can be solved using constant-speed straight-line motion. The idea is shown in Fig. 1.8.

Continuous-time model within one frame During one frame update, we assume constant velocity. Let the current ball center be (x_0, y_0) and velocity be (v_x, v_y) (pixels/frame). Then for the normalized time $t \in [0, 1]$ within a frame,

$$x(t) = x_0 + v_x t, \qquad y(t) = y_0 + v_y t. \tag{1.15}$$

To include the ball radius R using the mass-point idea, we treat the ball as a point and expand the brick boundary by R (same idea as "shrink the ball to a point, expand the obstacle by R"). Let the expanded brick boundaries be

$$x \in [L, R], \qquad y \in [B, T], \tag{1.16}$$

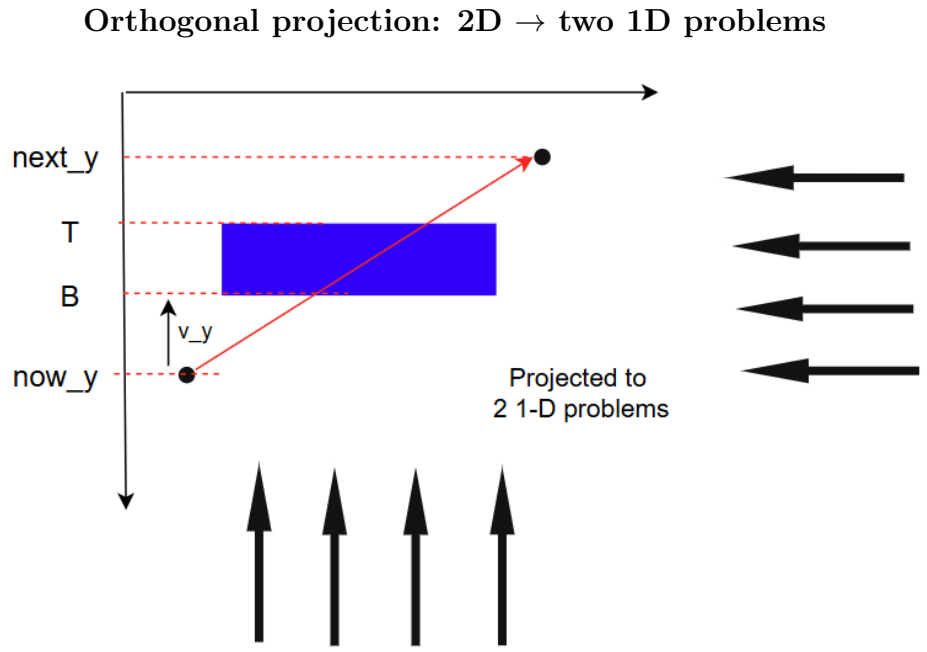


Fig. 1.8. Project the swept motion during one frame onto x- and y-axes.

where L, R, B, T denote the left, right, bottom, and top boundaries of the *expanded* brick rectangle.

1D time interval for y-projection We now show the y-direction handling method. We compute the time when the moving point reaches the two y-boundaries T and B:

$$t_T = \frac{T - y_0}{v_y}, \qquad t_B = \frac{B - y_0}{v_y}, \tag{1.17}$$

where the division is *signed* (so the formulas work for both $v_y > 0$ and $v_y < 0$). Then the entering and exiting times for the y-interval are

$$t_{y,\text{enter}} = \min\{t_T, t_B\}, \qquad t_{y,\text{exit}} = \max\{t_T, t_B\}. \tag{1.18}$$

The geometric meaning is illustrated in Fig. 1.9. **Notice:** since we only care about motion within one frame, the valid time range is $t \in [0, 1]$.
Similarly, for x-direction,

$$t_L = \frac{L - x_0}{v_x}, \qquad t_R = \frac{R - x_0}{v_x}, \qquad t_{x,\text{enter}} = \min\{t_L, t_R\}, \qquad t_{x,\text{exit}} = \max\{t_L, t_R\}. \tag{1.19}$$

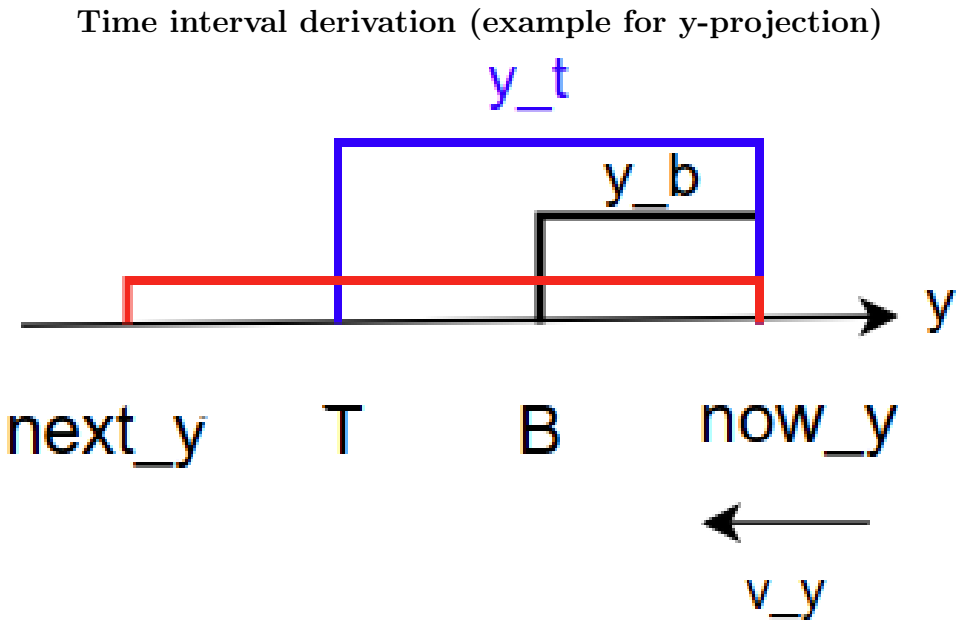


Fig. 1.9. Compute $[t_{y,\text{enter}},\, t_{y,\text{exit}}]$ by projecting motion onto the y-axis (similar for x-axis).

Collision determination by interval intersection Suppose we have obtained the two time intervals

$$I_x = [t_{x,\text{enter}},\, t_{x,\text{exit}}], \qquad I_y = [t_{y,\text{enter}},\, t_{y,\text{exit}}]. \tag{1.20}$$

The ball is inside the expanded brick during the times when *both* 1D conditions are satisfied, i.e., when $t \in I_x \cap I_y$. Therefore, a collision occurs if the intersection is non-empty within one frame:

$$I_x \cap I_y \cap [0, 1] \neq \emptyset. \tag{1.21}$$

Equivalently, define

$$t_{\text{enter}} = \max\{t_{x,\text{enter}},\, t_{y,\text{enter}}\}, \qquad t_{\text{exit}} = \min\{t_{x,\text{exit}},\, t_{y,\text{exit}}\}, \tag{1.22}$$

then collision happens if

$$t_{\text{enter}} \leq t_{\text{exit}} \quad \text{and} \quad t_{\text{exit}} \geq 0 \quad \text{and} \quad t_{\text{enter}} \leq 1. \tag{1.23}$$

This intersection idea is illustrated in Fig. 1.10.

Collision test by interval intersection

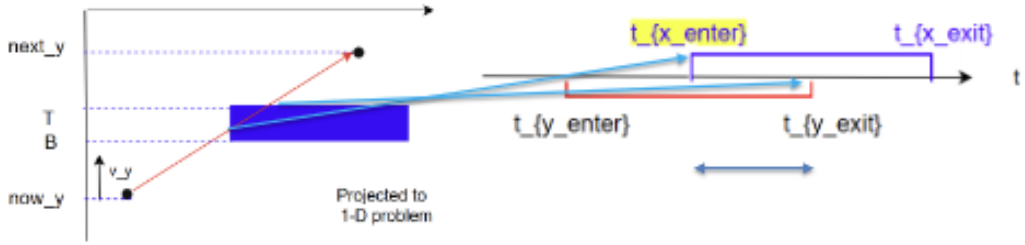


Fig. 1.10. A collision occurs if I_x and I_y overlap (non-empty intersection) within one frame.

Which axis is collided? (reflection axis) Under the assumption that a collision happens, the entering time indicates which boundary is hit first. Based on the projection logic, the *bigger entering time* determines the collision axis:

$$\begin{aligned} &\text{if } t_{x,\text{enter}} > t_{y,\text{enter}}, \text{ the collision normal is along x (hit left/right face);} \\ &\text{else, the collision normal is along y (hit top/bottom face).} \end{aligned} \tag{1.24}$$

Then we reverse the corresponding velocity component to realize reflection (mirror principle).

Thus, the two key problems in the brick and ball interaction—(1) how to determine there is a collision, and (2) how to know which axis the ball collides with—are answered by the projection-and-interval method.

Section 2.2.2 multiple bricks problem

Because we have tackled the single brick problem, we directly tried to apply the same logic to each brick, as shown below.

Naive idea (apply single-brick logic to all bricks). We check collision with every brick independently:

```
for each brick i = 1..N:
    if (brick_i detected):
        do reflection (same as single brick case)
        delete brick_i if needed
```

However, there is a trouble. Different from the single brick problem, in the multiple-brick scenario, a ball may go through many bricks between two neighbouring frames, which is shown below.

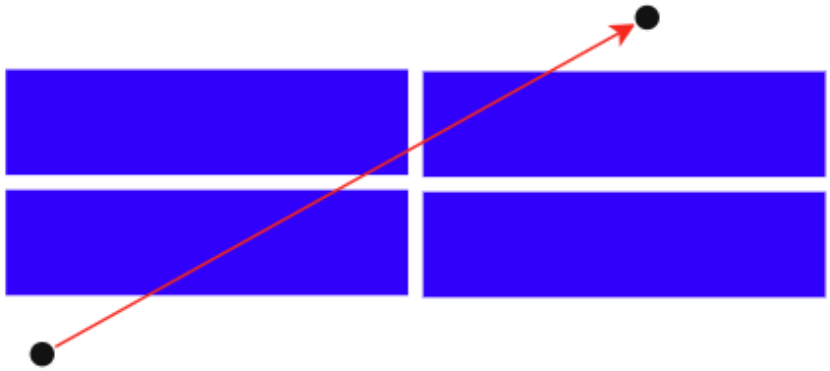


Fig. 1.11. Multiple-brick scenario: the ball may cross several bricks within one frame interval.

Solution (transform multiple to single). The solution is selecting the minimum entering time among all entering times from each single brick-ball collision. That is, we compute $(t_{\text{enter}}^{(i)}, t_{\text{exit}}^{(i)})$ for each brick i, keep only valid collisions, and choose

$$t_{\text{enter}}^{\star} = \min_i t_{\text{enter}}^{(i)}.$$

Then we only handle the brick that achieves $t_{\text{enter}}^{\star}$, and apply the same reflection rule as the single brick problem. This method transforms the multiple bricks problem into a single brick problem, as shown below.

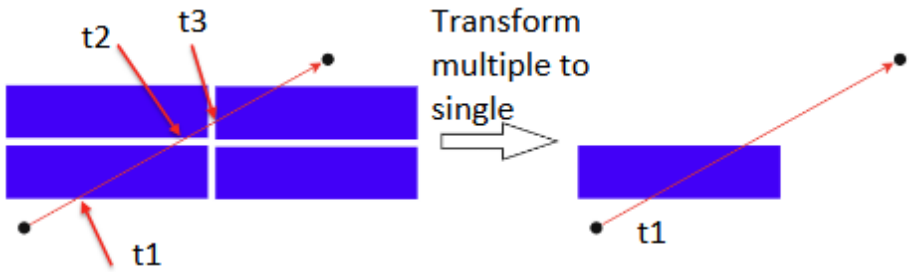


Fig. 1.12. Transform the multiple-brick case into a single-brick collision by choosing the earliest entering time.

Pseudo code (final strategy).

```
t_min  = +INF
hit_id = -1

for each brick i = 1..N:
    compute t_enter(i), t_exit(i) using the single-brick method
```

```
    if collision valid and t_enter(i) < t_min:
        t_min  = t_enter(i)
        hit_id = i

if hit_id != -1:
    do reflection using collision normal of brick hit_id
    delete brick hit_id
```

Section 3: Video with Functional Buttons

3.1 board vs walls

button-controlled board Designed Waveforms or Thoughts

We expect that as the user presses the functional button, the board will move with the same updating speed as the ball movement. The scenario figure is shown below. y_1 indicates the fixed y value of the board.

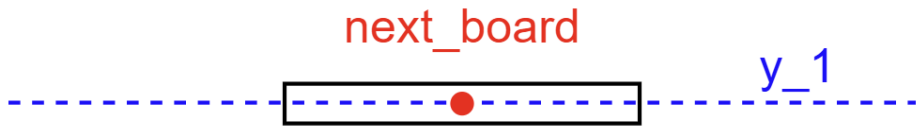


Fig. 1.13. Board motion scenario. The board moves horizontally at a fixed vertical position y_1.

In detail, based on the following formulation, considering the x coordinate of the center of the board, the user will control the sign of the board velocity, provided a fixed speed size (indicated as constant A).

$$x_{\text{board}}^{\text{next}} = x_{\text{board}}^{\text{now}} + \mathbf{v_x}. \tag{1.25}$$

Here, $\mathbf{v_x}$ is determined by button detection:

$$\mathbf{v_x} = \begin{cases} +A, & \text{pressed right button,} \\ -A, & \text{pressed left button,} \\ 0, & \text{no button pressed.} \end{cases}$$

The overall logic is shown below.

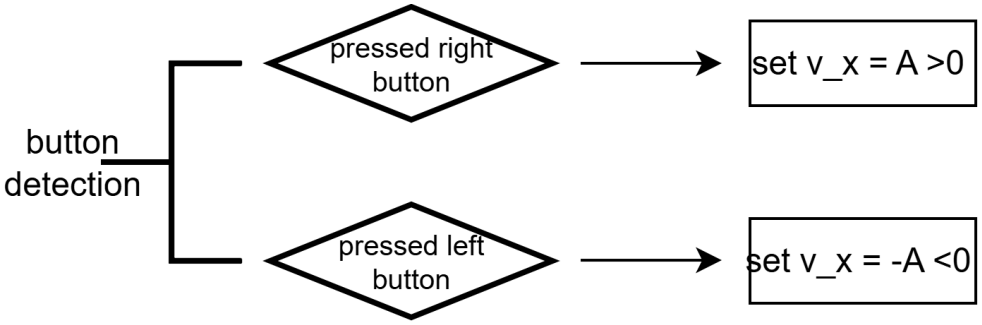


Fig. 1.14. Button-controlled board logic: set $\mathbf{v_x}$ according to left/right button detection.

However, because of the mechanical button issue, we need to do debounce with help of a counter. Intuitively, the logic is: when the button is pressed for a long time, the button is actually pressed. Besides the pressed case, we also need to do debounce for releasing a button. The following waveform shows the debounce effect from `button` (a direct mechanical button) to `detection_result` (a no-jitter button).

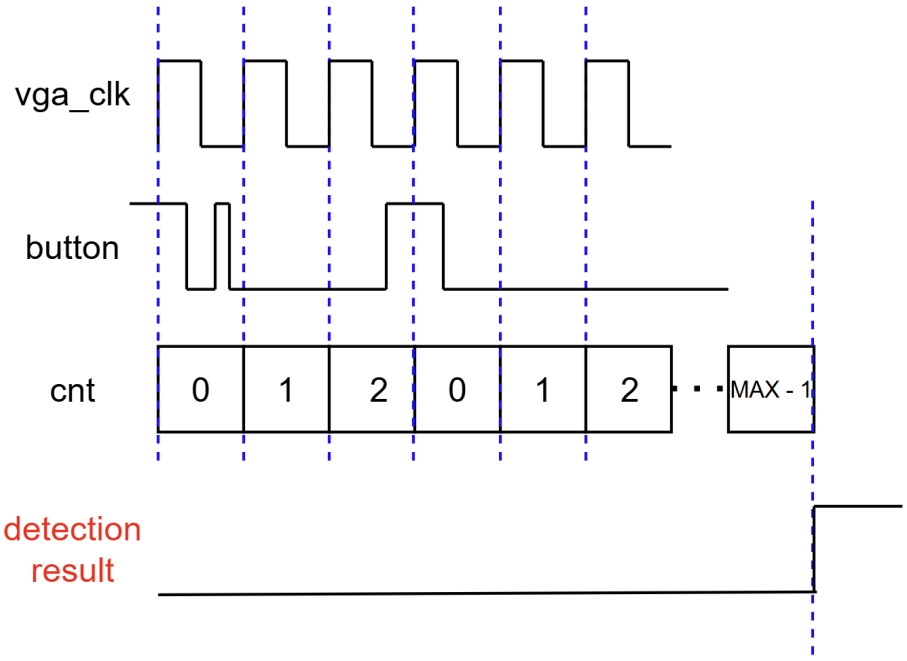


Fig. 1.15. Debounce waveform: from mechanical `button` to stable `detection_result` using a counter `cnt`.

Explanation for the debounce principle of press case.

Ⓐ Check whether `button` is high or low at each rising edge of the clock. If low (meaning pressed), then the counter is increased by 1.

Ⓑ Continuously check for the following clocks, until the counter value achieves `MAX - 1`. The `MAX` is from our setting, corresponding to the "when the button is pressed for a long time" as stated before.

Ⓒ If the counter value is `MAX - 1`, then at the next check time point, the detection result (no-jitter button) will change to activation, meaning pressed.

Note: You may notice in the middle of the figure, the `cnt` value changes from 2 to 0. That is the effect of the button debounce logic. Because at the rising edge, the jitter phenomenon makes the mechanical button be the released level, although the user is pressing the button.

board constrained by wall Designed Waveforms or Thoughts

This section is simple. The task we face is required that the board cannot move out of the screen. In formulation, we clamp the next board center position $x_{\text{board}}^{\text{next}}$ into a valid range.

Formulation. Let L and R denote the left and right walls (in x-coordinate), and let ℓ_{half} be the half length of the board. Then the valid range of the board center is:

$$L + \ell_{\text{half}} \;\le\; x_{\text{board}} \;\le\; R - \ell_{\text{half}}.$$

Update rule (clamping). After computing the unconstrained update $x_{\text{board}}^{\text{next}}$, we apply:

$$x_{\text{board}}^{\text{next}} = \begin{cases} R - \ell_{\text{half}}, & \text{if } x_{\text{board}}^{\text{next}} > R - \ell_{\text{half}}, \\ L + \ell_{\text{half}}, & \text{if } x_{\text{board}}^{\text{next}} < L + \ell_{\text{half}}, \\ x_{\text{board}}^{\text{next}}, & \text{otherwise.} \end{cases} \tag{1.26}$$

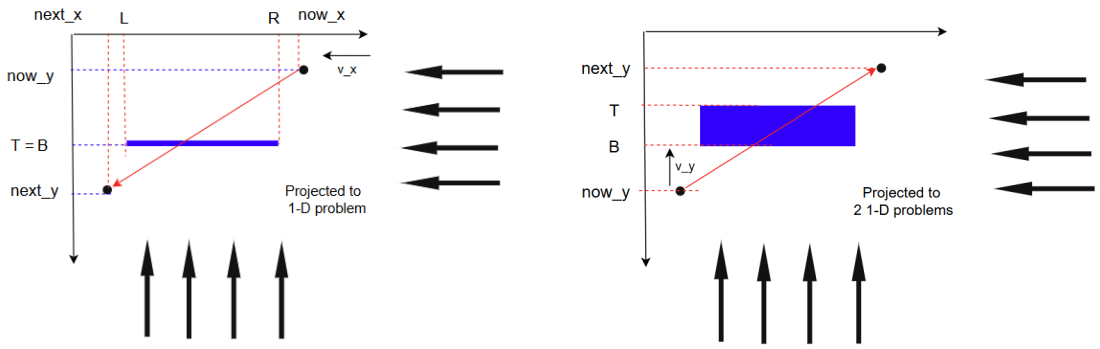
Pseudo code.

```
x_next = x_next_unclamped

if (x_next > R - half_len)  x_next = R - half_len
if (x_next < L + half_len)  x_next = L + half_len
```

3.2 ball vs board Designed Waveforms or Thoughts

Because we have solved the ball and brick problem, by treating the board as a movable brick, we can reuse the collision detection and reflection logic. From the figure below, you can see the similarity between the two problems.



(a) Ball vs board scenario. (b) Ball vs brick scenario.

Fig. 1.16. Similarity between ball–board and ball–brick collision detection (projection to 1-D / two 1-D problems).

For the ball and brick problem, from the resulted time interval intersection view, there are two possibilities if there is a collision, which are shown below. We only need to implement the second one, top and bottom case, and the bottom case is useless, so we can only implement the top case.

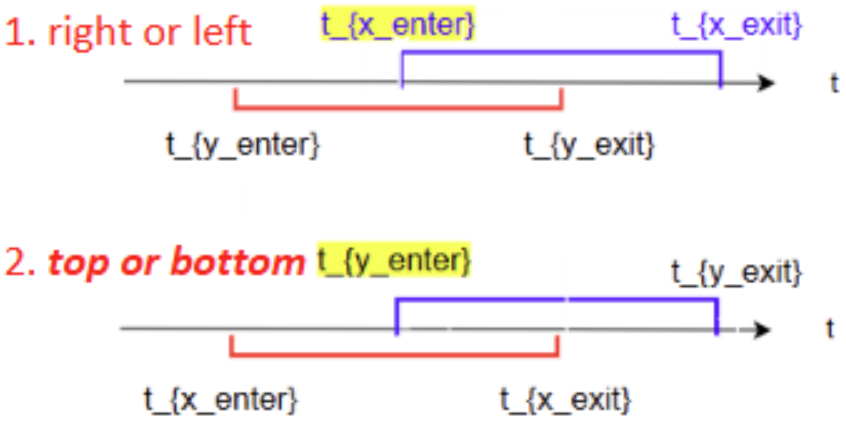


Fig. 1.17. Time-interval intersection view: two possible collision normals (left/right vs top/bottom). For ball vs board, we only keep the top collision case.

But some modifications are required: only top collisions are valid; bottom collisions are impossible; and side collisions are meaningless. In below, we use the scenario figure to show the reasons.

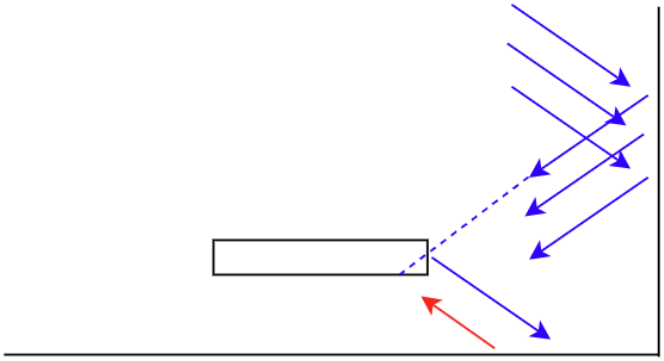


Fig. 1.18. Ball vs board: only top collision is considered; bottom collision is impossible and side collision is meaningless in our setting.

Ⓐ **For bottom collisions of board.**
The red arrow shows the situation. We show it is impossible by contradiction. Suppose the bottom collision happens, then the ball needs to be reflected by the bottom wall. However, we have set the bottom wall cannot be used to reflect the ball. So, the initial assumption is false. Thus, the bottom collision of board is impossible to happen.
For the design side, the result is: we do not need to consider the bottom collision of the board. This simplicity is ensured by the logic derivation shown above.

Ⓑ **For side collisions of board.**
The blue arrows show the situation. We show it is meaningless by assuming we have the side collision and reflection mechanism. From the figure, if the ball has a collision along the dashed line, it will be reflected to the right downward direction. The direction will immediately result the ball into a die. So, it is meaningless.
For simplicity, in implementation, we will not consider the side collision. The simplicity is ensured by our own setting, but it is reasonable.

Fig. 1.19

1.2.3 Prepare FPGA Solution for Page Controllers

Because the system can be seen as a combination of game and some static pages, and there are also some logic control inside the game process, e.g. when the ball is out of the bottom side of screen, the ball will be reset at the initial position, and the bricks and the board will not be reset.
As a result, we used two FSMs to manage the whole system.

The overview of the two FSMs logic is:

Ⓐ When the system begins, the user gets into the START page, corresponding to the START state of Main FSM. If the user presses the button, the Main FSM will change the state, and get into the Main Game state.

Ⓑ Now, getting into the Main Game page, the Main FSM will deliver a signal called `new_game` to the Sub FSM. The effect is to make the Sub FSM begin to work.

Ⓒ Some details of Sub FSM working logic are: (1) if there is no bricks, it will deliver a signal called `N_B` to Main FSM to make the system get into the CLEAR state; (2) if there is no health points, it will deliver a signal called `N_H_P` to Main FSM to make the system get into the GAME OVER state; (3) if the ball is out of the bottom wall, then the Sub FSM will reset ball, and keep bricks and board.

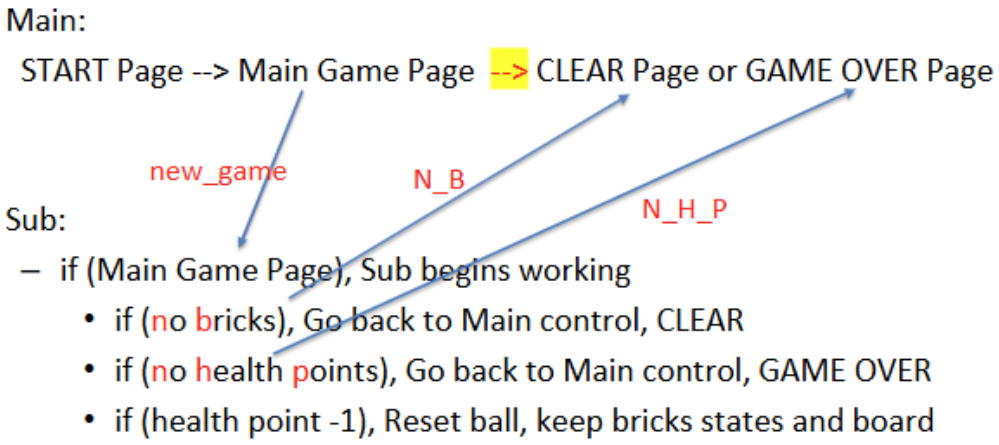


Fig. 1.20. Overview of the two-FSM control logic (Main FSM + Sub FSM).

Main Controller (Main FSM)

The details of Main FSM design are shown below.

Table 1.1: Main FSM: Inputs, Outputs, and States.

Inputs	
	– `btn`
	– `N_H_P`: No Health Points
	– `N_B`: No Bricks
Outputs	
	– `frame`: determine the page (START / MAIN_GAME / CLEAR / GAME_OVER)
	– `new_game = 1`: tell to the Sub FSM "You should work"
States	`START`, `MAIN_GAME`, `CLEAR`, `GAME_OVER`

And the state transition diagram is shown below.

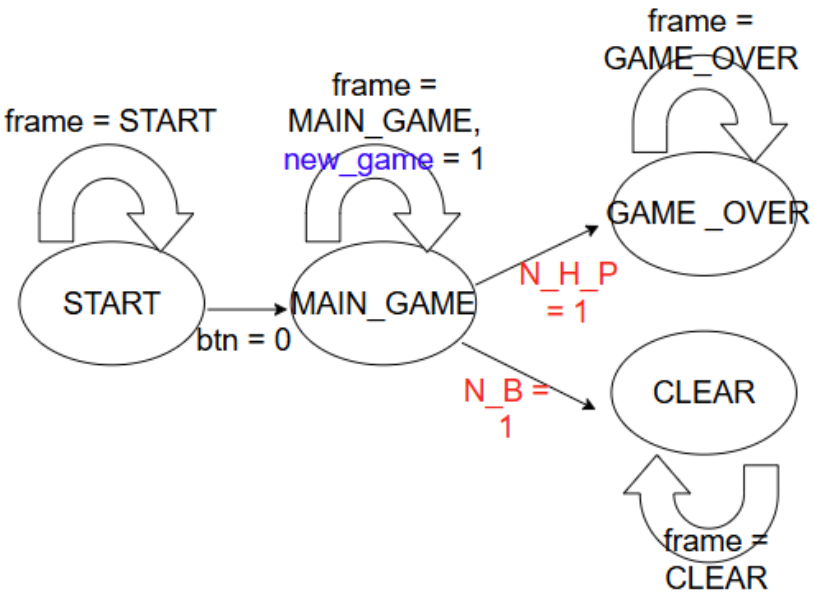


Fig. 1.21. State transition diagram of the Main FSM.

Notice: On reset (`rstn = 0`), the FSM immediately returns to the `START` state from any state.

Sub Controller (Sub FSM)

The details of Sub FSM design are shown below.

Table 1.2: Sub FSM: Inputs, Outputs, and States.

Inputs	
	– `new_game`: from Main FSM
	– `ball_alive`
	– `no_bricks`
Outputs	
	– `life_level`: {3, 2, 1, 0} (die)
	– `rst_ball`: if lose one life
	– `N_H_P`
	– `N_B`
States	`3L`, `2L`, `1L`, `DIE`, `CLEAR`

The state transition diagram is shown below.

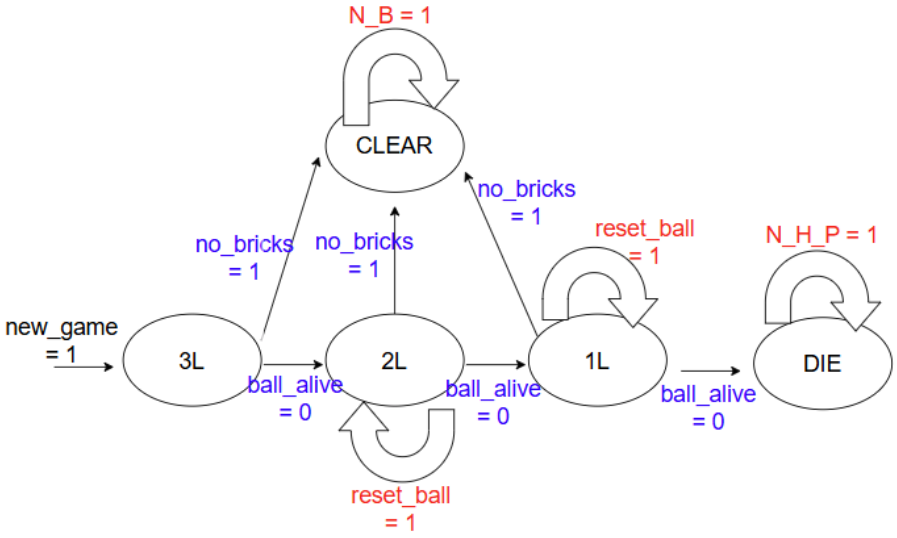


Fig. 1.22. State transition diagram of the Sub FSM.

Notice: `new_game` is like a local reset. Once it is 1, the Sub FSM begins to work.

1.2.4 Prepare FPGA Solution for Static Pages

START

GAME OVER

CLEAR

1.2.5 Prepare FPGA Solution for Dynamic Pages

Three Health Points Rendering and LED

Rendering Priority

In dynamic page, a challenge is the overlap for two objects. One example is from the overlap between ball and hearts (health points indicator), which is shown below.

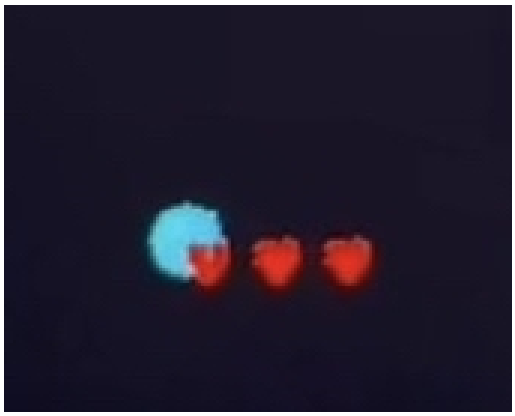


Fig. 1.23. An example of overlap: ball and hearts (health points indicator) may appear at the same pixels.

In logic, the overlap is from that conditions may be simultaneously satisfied, which is shown in the flowchart below.

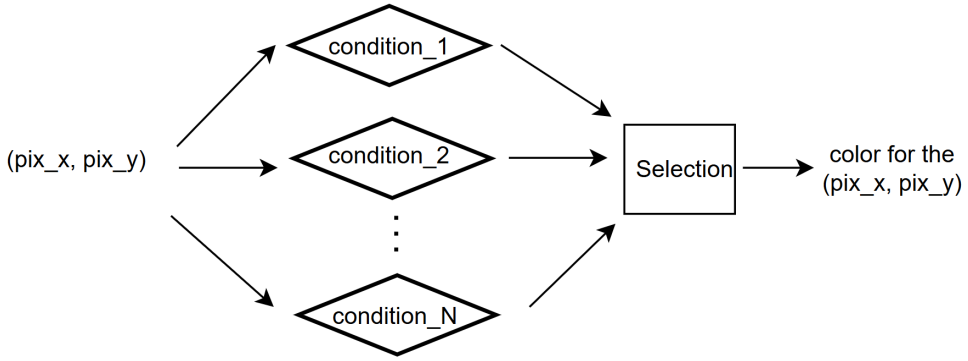


Fig. 1.24. Dynamic rendering logic: multiple conditions may be true for the same $(\texttt{pix\_x}, \texttt{pix\_y})$, then a selection is required.

The solution is setting the rendering priority for the different objects. In the above example, the priority of hearts (HP) is greater than ball.
In fact, the priority will be ensured by some `if--else` structure at the code level, e.g.

```
if (is_heart_pixel)      color = HEART_COLOR;
else if (is_ball_pixel) color = BALL_COLOR;
else                     color = BACKGROUND_COLOR;
```

1.3 Implementations

1.3.1 For Game Process

Following the design steps, we will implement 1. wall and ball, 2. ball and single brick (based on 1), 3. button-controlled board constrained by wall, 4. game process with single brick, 5. game process with multiple bricks.
The fifth module serves as a sublevel module for the whole Breakout game system, which will be used for further combination with page controllers and static pages.

(1) Wall and Ball: `vga_pic_move2`

Github:
`Reproduce_ExampleGame/Part2_Video_Without_Buttons/section1_reflectionByWalls`
This module achieves the ball movement with reflection from the walls, and renders the ball on the VGA screen. It is the first functional module in our implementation pipeline, providing a correct and stable baseline for later extensions (e.g., bricks, board, and full game process).

Module interface. The module `vga_pic_move2` takes the VGA pixel coordinates `pix_x`, `pix_y` (signed), and outputs the 16-bit RGB565 pixel color `pix_data`. The update clock is `vga_clk` (25 MHz), and `sys_rst_n` is an active-low reset. The input `up` is reserved and unused at this stage.

Parameters and state variables. We follow the standard visible region configuration `H_VALID` = 640, `V_VALID` = 480. The ball is represented by its center position $(x_{\text{ball}}, y_{\text{ball}})$ and a fixed radius `BALL_RADIUS`. The movement is defined by signed velocities Δx and Δy, stored as `delta_x` and `delta_y`. Two constant colors are used: ball color and background color.

Frame-based update mechanism. To obtain smooth and stable animation, the ball position is updated only once per frame. We define a `frame_tick` when scanning reaches the last pixel of the visible region:

$$\texttt{frame\_tick} \iff (\texttt{pix\_y} = \texttt{V\_VALID} - 1) \ \wedge \ (\texttt{pix\_x} = \texttt{H\_VALID} - 1).$$

Thus, the ball moves at a frame rate rather than at the pixel clock rate.

Boundary definition and signed-width extension. In our code, the next-step position is computed by widened signed arithmetic:

$$x_{\text{next}} = x_{\text{ball}} + \Delta x, \qquad y_{\text{next}} = y_{\text{ball}} + \Delta y.$$

To avoid overflow and keep comparison consistent, we implement them as `x_next_s` and `y_next_s` with 12-bit signed wires: `x_next_s = ball_x + delta_x`, `y_next_s = ball_y + delta_y`.
Meanwhile, the valid boundaries are defined with the ball radius:

$$X_{\min} = r, \quad X_{\max} = H_{\text{VALID}} - r, \qquad Y_{\min} = r, \quad Y_{\max} = V_{\text{VALID}} - r,$$

where $r =$ `BALL_RADIUS`. In the implementation, these are `X_MIN, X_MAX, Y_MIN, Y_MAX`.

Position update with clamping (frame tick). At each `frame_tick`, we update the ball center by clamping the next-step result into the valid range. This guarantees that the ball never moves out of the screen even when the step size is relatively large:

$$x_{\text{ball}} \leftarrow \text{clamp}(x_{\text{next}}, X_{\min}, X_{\max}), \qquad y_{\text{ball}} \leftarrow \text{clamp}(y_{\text{next}}, Y_{\min}, Y_{\max}).$$

In code, this is achieved by two conditional assignments to `ball_x` and `ball_y` based on `x_next_s` and `y_next_s`.

Wall reflection by velocity reversal. The wall reflection is implemented by reversing the sign of velocity when the next-step value reaches the boundary:

$$\text{if } (x_{\text{next}} \le X_{\min} \text{ or } x_{\text{next}} \ge X_{\max}) \Rightarrow \Delta x \leftarrow -\Delta x,$$

$$\text{if } (y_{\text{next}} \le Y_{\min} \text{ or } y_{\text{next}} \ge Y_{\max}) \Rightarrow \Delta y \leftarrow -\Delta y.$$

A key point in our implementation is that both the clamping and the velocity flip are based on the same intermediate wires `x_next_s` and `y_next_s`, which avoids inconsistent updates caused by recomputing conditions in multiple places.

Pixel rendering (robust in-circle test). For each scanned pixel (`pix_x`, `pix_y`), we determine whether it belongs to the ball region via a squared-distance test:

$$(\texttt{pix\_x} - x_{\text{ball}})^2 + (\texttt{pix\_y} - y_{\text{ball}})^2 \le r^2.$$

In the code, we compute dx and dy using widened signed arithmetic, then square them to dx2 and dy2, and compare dist2 with R2. Finally,

$$\texttt{pix\_data} = \begin{cases} \texttt{BALL\_COLOR}, & \texttt{dist2} \le \texttt{R2}, \\ \texttt{BACKGROUND\_COLOR}, & \text{otherwise.} \end{cases}$$

Summary of this module. This module provides a complete minimal dynamic VGA scene: (1) frame-synchronized position update, (2) correct wall reflection via velocity reversal, and (3) robust circle rendering via squared-distance comparison. It serves as the baseline for the subsequent modules (single brick, board control, and full game process).

Full Verilog code. The complete implementation of vga_pic_move2 is shown below.

```
`timescale 1ns/1ps

module vga_pic_move2( // this file achieves the ball movement
    with reflection from the walls
    input  wire         vga_clk,      // 25 MHz
    input  wire         sys_rst_n,    // active-low
    input  wire signed [9:0]  pix_x,
    input  wire signed [9:0]  pix_y,
    input  wire         up,           // unused for now
    output reg  [15:0] pix_data
);

    // ===== params =====
    localparam [9:0]  H_VALID = 10'sd640;
    localparam [9:0]  V_VALID = 10'sd480;

    localparam [15:0] BLUE    = 16'h001F;
    localparam [15:0] PURPPLE = 16'hF81F;

    // ball state
    reg signed [9:0] ball_x = 10'sd320;
    reg signed [9:0] ball_y = 10'sd240;
    localparam [9:0]  BALL_RADIUS      = 10'sd10;
    localparam [15:0] BALL_COLOR       = BLUE;
    localparam [15:0] BACKGROUND_COLOR = PURPPLE;
```

```
25
26      // signed velocity
27      reg signed [9:0] delta_x = 10'sd25; // 12
28      reg signed [9:0] delta_y = 10'sd45; // 22
29
30      // widened next-step computation (avoid overflow)
31      wire signed [11:0] x_next_s = $signed({1'b0, ball_x}) +
           $signed(delta_x);
32      wire signed [11:0] y_next_s = $signed({1'b0, ball_y}) +
           $signed(delta_y);
33
34      localparam signed [11:0] X_MIN = $signed({1'b0,
           BALL_RADIUS});
35      localparam signed [11:0] X_MAX = $signed({1'b0, H_VALID
           - BALL_RADIUS});
36      localparam signed [11:0] Y_MIN = $signed({1'b0,
           BALL_RADIUS});
37      localparam signed [11:0] Y_MAX = $signed({1'b0, V_VALID
           - BALL_RADIUS});
38
39      // for smooth vision: update once per frame
40      wire frame_tick = (pix_y == V_VALID - 1) && (pix_x ==
           H_VALID - 1);
41
42      // ===== position & velocity update in clock domain =====
43      always @(posedge vga_clk or negedge sys_rst_n) begin
44          if (!sys_rst_n) begin
45              ball_x  <= 10'sd320;
46              ball_y  <= 10'sd240;
47              delta_x <= 10'sd25; // 50
48              delta_y <= 10'sd45; // 90
49          end else if (frame_tick) begin
50
51              // position clamp to keep ball inside the screen
52              ball_x <= (x_next_s <= X_MIN) ? X_MIN[9:0] :
53                        (x_next_s >= X_MAX) ? X_MAX[9:0] :
54                                              x_next_s[9:0];
55
56              ball_y <= (y_next_s <= Y_MIN) ? Y_MIN[9:0] :
57                        (y_next_s >= Y_MAX) ? Y_MAX[9:0] :
```

```
58                                                  y_next_s[9:0];
59
60              // velocity reversal (wall reflection)
61              if (x_next_s <= X_MIN || x_next_s >= X_MAX)
                   delta_x <= -delta_x;
62              if (y_next_s <= Y_MIN || y_next_s >= Y_MAX)
                   delta_y <= -delta_y;
63
64          end
65      end
66
67      // ===== robust in-circle test =====
68      // widen and use signed subtraction, then square (avoid
           overflow)
69      wire signed [11:0] dx = $signed({1'b0, pix_x}) - $signed
           ({1'b0, ball_x});
70      wire signed [11:0] dy = $signed({1'b0, pix_y}) - $signed
           ({1'b0, ball_y});
71      wire [23:0] dx2 = dx * dx;
72      wire [23:0] dy2 = dy * dy;
73      wire [24:0] dist2 = dx2 + dy2;
74      localparam [24:0] R2 = 25'd10 * 25'd10;
75
76      always @* begin
77          pix_data = (dist2 <= R2) ? BALL_COLOR :
               BACKGROUND_COLOR;
78      end
79
80  endmodule
```

Listing 1.1: vga_pic_move2: ball movement with wall reflection.

(2) Single Brick and Ball

Github

/Reproduce_ExampleGame/Part2_Video_Without_Buttons/section2_effectsFromBricks/vga_pic_move2.v

This module extends the baseline "wall–ball" motion by adding one brick, and implements brick–ball collision detection, ball reflection, and brick deletion. It follows the design idea in Section 2.2.1: transform the disk–rectangle collision into

a point–rectangle collision by expanding the rectangle, and detect collision within one frame by time-interval intersection.

Core implementation idea. A direct frame-to-frame overlap test may miss collisions when the ball moves fast. Therefore, we use a swept (continuous-time) collision detection within one frame interval. Let the ball center at the beginning of a frame be (x_0, y_0), and the velocity be (v_x, v_y). For $t \in [0, 1]$,

$$x(t) = x_0 + v_x t, \qquad y(t) = y_0 + v_y t.$$

To handle ball radius r, we expand the brick boundaries:

$$L' = L - r, \quad R' = R + r, \quad T' = T - r, \quad B' = B + r,$$

so that the original disk–rectangle collision becomes a point–expanded-rectangle collision.

Time-interval intersection (swept AABB). We compute entering/exiting time intervals on each axis:

$$I_x = [t_{x,\text{enter}}, t_{x,\text{exit}}], \qquad I_y = [t_{y,\text{enter}}, t_{y,\text{exit}}],$$

and then

$$t_{\text{enter}} = \max(t_{x,\text{enter}}, t_{y,\text{enter}}), \qquad t_{\text{exit}} = \min(t_{x,\text{exit}}, t_{y,\text{exit}}).$$

A collision occurs within one frame if

$$t_{\text{enter}} \le t_{\text{exit}}, \qquad 0 \le t_{\text{enter}} \le 1.$$

In hardware, we implement t using fixed-point arithmetic with `SCALE=256` (i.e., $t \in [0, 1]$ is mapped to $[0, 256]$). We also use `inside0` to suppress the false hit when the start point is already inside the expanded rectangle.

Reflection axis decision. We determine the collision normal by comparing the two entering times:

if $t_{x,\text{enter}} > t_{y,\text{enter}}$, hit left/right face and flip v_x; else, hit top/bottom face and flip v_y.

This corresponds to `hit_axis_x_w` in code.

Frame update and brick deletion. All state updates are performed only at `frame_tick` (end of visible scanning), so the motion updates once per frame. When `brick_alive && hit_w` is true, we:

Ⓐ Move the ball center to the contact point $(x_0 + v_x t_{\text{enter}},\ y_0 + v_y t_{\text{enter}})$.

Ⓑ Flip exactly one velocity component (v_x or v_y) according to the collision normal.

Ⓒ Add a ± 1 pixel offset along the reflected axis to avoid sticking on the surface due to discrete-time sampling.

Ⓓ Set `brick_alive <= 1'b0`, so the brick disappears in both logic and rendering.

If there is no brick collision in the current frame, we fall back to the wall-reflection logic.

Full Verilog listing. For completeness, the full implementation of `vga_pic_move2` is shown below.

```
`timescale 1ns/1ps
module vga_pic_move2(// this module implements one brick
   effect
    input  wire         vga_clk,      // 25 MHz
    input  wire         sys_rst_n,    // active-low
    input  wire  signed [9:0]  pix_x,
    input  wire  signed [9:0]  pix_y,
    input  wire         up,           // unused for now
    output reg  [15:0] pix_data
);

    // ===== params =====
    localparam [9:0]  H_VALID = 10'sd640;
    localparam [9:0]  V_VALID = 10'sd480;

    localparam [15:0] BLUE    = 16'h001F;
    localparam [15:0] PURPPLE = 16'hF81F;
    localparam [15:0] RED     = 16'hF800;

    // ball state
    reg signed [9:0] ball_x = 10'sd320;
```

```
21      reg signed [9:0] ball_y = 10'sd240;
22      localparam [9:0]  BALL_RADIUS      = 10'sd10;
23      localparam [15:0] BALL_COLOR       = BLUE;
24      localparam [15:0] BRICK_COLOR      = 16'hFFFF;
25      localparam [15:0] BACKGROUND_COLOR = PURPPLE;
26
27      // signed velocity
28      reg signed [9:0] delta_x = 10'sd25;
29      reg signed [9:0] delta_y = 10'sd45;
30
31      // next position & wall bounds
32      wire signed [11:0] x_next_s = $signed({1'b0, ball_x}) +
           $signed(delta_x);
33      wire signed [11:0] y_next_s = $signed({1'b0, ball_y}) +
           $signed(delta_y);
34
35      localparam signed [11:0] X_MIN = $signed({1'b0,
           BALL_RADIUS});
36      localparam signed [11:0] X_MAX = $signed({1'b0, H_VALID
           - BALL_RADIUS});
37      localparam signed [11:0] Y_MIN = $signed({1'b0,
           BALL_RADIUS});
38      localparam signed [11:0] Y_MAX = $signed({1'b0, V_VALID
           - BALL_RADIUS});
39
40      // one brick
41      reg brick_alive = 1'b1;
42      localparam signed [9:0] L = 10'sd200, R = 10'sd260; //
           width 60
43      localparam signed [9:0] T = 10'sd150, B = 10'sd170; //
           height 20
44
45      // === swept method ===
46      localparam integer SCALE = 256;
47      localparam signed [15:0] INF_POS = 16'sh7FFF;
48      localparam signed [15:0] INF_NEG = -16'sh7FFF;
49
50      wire signed [11:0] x0 = ball_x, y0 = ball_y;
51      wire signed [11:0] vx = delta_x, vy = delta_y;
52      wire signed [11:0] r  = BALL_RADIUS;
```

```
53
54      // expanded rectangle
55      wire signed [11:0] Lp = L - r;
56      wire signed [11:0] Rp = R + r;
57      wire signed [11:0] Tp = T - r;
58      wire signed [11:0] Bp = B + r;
59
60      wire inside0 = (x0 >= Lp) && (x0 <= Rp) && (y0 >= Tp) &&
           (y0 <= Bp);
61
62      wire signed [23:0] t1x_fp_w = (vx==0) ? 24'sd0 : ( (Lp -
           x0) * SCALE ) / vx;
63      wire signed [23:0] t2x_fp_w = (vx==0) ? 24'sd0 : ( (Rp -
           x0) * SCALE ) / vx;
64      wire signed [23:0] t1y_fp_w = (vy==0) ? 24'sd0 : ( (Tp -
           y0) * SCALE ) / vy;
65      wire signed [23:0] t2y_fp_w = (vy==0) ? 24'sd0 : ( (Bp -
           y0) * SCALE ) / vy;
66
67      wire signed [15:0] tx_enter_fp_w =
68          (vx==0) ? ((x0 < Lp || x0 > Rp) ? INF_POS : INF_NEG)
69                  : ( (t1x_fp_w < t2x_fp_w) ? t1x_fp_w[15:0] :
                       t2x_fp_w[15:0] );
70      wire signed [15:0] tx_exit_fp_w  =
71          (vx==0) ? ((x0 < Lp || x0 > Rp) ? INF_NEG : INF_POS)
72                  : ( (t1x_fp_w < t2x_fp_w) ? t2x_fp_w[15:0] :
                       t1x_fp_w[15:0] );
73
74      wire signed [15:0] ty_enter_fp_w =
75          (vy==0) ? ((y0 < Tp || y0 > Bp) ? INF_POS : INF_NEG)
76                  : ( (t1y_fp_w < t2y_fp_w) ? t1y_fp_w[15:0] :
                       t2y_fp_w[15:0] );
77      wire signed [15:0] ty_exit_fp_w  =
78          (vy==0) ? ((y0 < Tp || y0 > Bp) ? INF_NEG : INF_POS)
79                  : ( (t1y_fp_w < t2y_fp_w) ? t2y_fp_w[15:0] :
                       t1y_fp_w[15:0] );
80
81      wire signed [15:0] t_enter_fp_w = (tx_enter_fp_w >
           ty_enter_fp_w) ? tx_enter_fp_w : ty_enter_fp_w;
82      wire signed [15:0] t_exit_fp_w  = (tx_exit_fp_w  <
```

```
    ty_exit_fp_w)  ? tx_exit_fp_w  : ty_exit_fp_w;

wire hit_w        = (!inside0) && (t_enter_fp_w <= t_exit_fp_w) &&
                    (t_enter_fp_w >= 0) && (t_enter_fp_w <= SCALE);
wire hit_axis_x_w = (tx_enter_fp_w > ty_enter_fp_w);

wire frame_tick = (pix_y == V_VALID - 1) && (pix_x == H_VALID - 1);

// ===== update =====
always @(posedge vga_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        ball_x      <= 10'sd320;
        ball_y      <= 10'sd240;
        delta_x     <= 10'sd25;
        delta_y     <= 10'sd45;
        brick_alive <= 1'b1;
    end else if (frame_tick) begin
        if (brick_alive && hit_w) begin
            ball_x <= x0 + ( (vx * t_enter_fp_w) >>> 8 );
            ball_y <= y0 + ( (vy * t_enter_fp_w) >>> 8 );

            if (hit_axis_x_w) begin
                delta_x <= -vx;
                ball_x  <= x0 + ( (vx * t_enter_fp_w) >>> 8 )
                              + ( (vx>=0) ? 10'sd1 : -10'sd1 );
            end else begin
                delta_y <= -vy;
                ball_y  <= y0 + ( (vy * t_enter_fp_w) >>> 8 )
                              + ( (vy>=0) ? 10'sd1 : -10'sd1 );
            end

            brick_alive <= 1'b0;

```

```
115             end else begin
116                 ball_x <= (x_next_s <= X_MIN) ? X_MIN[9:0] :
117                           (x_next_s >= X_MAX) ? X_MAX[9:0] :
118                                                 x_next_s
                                                       [9:0];
119
120                 ball_y <= (y_next_s <= Y_MIN) ? Y_MIN[9:0] :
121                           (y_next_s >= Y_MAX) ? Y_MAX[9:0] :
122                                                 y_next_s
                                                       [9:0];
123
124                 if (x_next_s <= X_MIN || x_next_s >= X_MAX)
                       delta_x <= -delta_x;
125                 if (y_next_s <= Y_MIN || y_next_s >= Y_MAX)
                       delta_y <= -delta_y;
126             end
127         end
128     end
129
130     // ===== render =====
131     wire signed [11:0] dx = $signed({1'b0, pix_x}) - $signed
           ({1'b0, ball_x});
132     wire signed [11:0] dy = $signed({1'b0, pix_y}) - $signed
           ({1'b0, ball_y});
133     wire [23:0] dx2 = dx * dx;
134     wire [23:0] dy2 = dy * dy;
135     wire [24:0] dist2 = dx2 + dy2;
136     localparam [24:0] R2 = 25'd10 * 25'd10;
137
138     wire in_brick = brick_alive &&
139                     (pix_x >= L) && (pix_x < R) &&
140                     (pix_y >= T) && (pix_y < B);
141
142     always @* begin
143         if (dist2 <= R2)
144             pix_data = BALL_COLOR;
145         else if (in_brick)
146             pix_data = BRICK_COLOR;
147         else
148             pix_data = BACKGROUND_COLOR;
```

```
149     end
150 endmodule
```

Listing 1.2: `vga_pic_move2.v`: single brick collision, reflection, and deletion

(3) button-controlled board constrained by wall

Github:

`Reproduce_ExampleGame/Part3_VideoWithButtons/warmup_board`

This module implements a horizontally movable board controlled by two mechanical buttons, while ensuring the board never moves out of the visible screen. At the implementation level, we decompose the function into three sublevel modules: (1) button debounce, (2) board movement control (with speed limiting and boundary clamp), and (3) rendering. Finally, we provide a top module that instantiates these modules and connects them to the VGA pipeline.

Module decomposition (implementation view).

Ⓐ **Debounce module `pressButton`.** Input: raw mechanical button `up`. Output: stable flag `key_up` (no jitter).

Ⓑ **Movement control module `boardMoveCtl`.** Input: two debounced button states. Output: board center position `x_board`.

Ⓒ **Rendering module `vga_pic_move_board`.** Input: pixel coordinate (`pix_x`, `pix_y`) and `x_board`. Output: pixel color `pix_data`.

(1) Button debounce: `pressButton`. At the code level, the debounce logic is implemented using **two independent counters**: one counter confirms a stable *press* (active-low), and the other confirms a stable *release*. Only when the corresponding counter reaches the threshold `CNT_20MS_MAX`, the stable output flag `key_up_flag` is updated. This ensures that short jitters do not trigger any false action.

The key code structure is:

- **Press counter:** counts while `up==0` (pressed), resets while `up==1`.
- **Release counter:** counts while `up==1` (released), resets while `up==0`.
- **Stable output update:** when either counter reaches `CNT_20MS_MAX-1`, `key_up_flag` switches to the corresponding stable state.

The following excerpt shows the stable flag update mechanism (quoted from the implementation):

```
always @(posedge vga_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        key_up_flag <= 1'b1;
    end else if (cnt_press_20ms >= CNT_20MS_MAX - 1'd1) begin
        key_up_flag <= 1'b0;
    end else if (cnt_release_20ms >= CNT_20MS_MAX - 1'd1)
       begin
        key_up_flag <= 1'b1;
    end else begin
        key_up_flag <= key_up_flag;
    end
end

assign key_up = key_up_flag;
```

(2) Board movement control: `boardMoveCtl`. This module converts two debounced buttons into a continuous board movement and keeps it inside the screen. The code-level implementation corresponds to three key actions:

Ⓐ **Debounced press interpretation (left/right exclusivity).** In our implementation, `pressButton` outputs `B_OK` where `0` means "stable pressed" and `1` means "stable released". Then we define a single-direction press (avoid conflicting commands):

```
wire left_pressed  = (B1_OK == 1'b0) && (B2_OK == 1'b1);
wire right_pressed = (B2_OK == 1'b0) && (B1_OK == 1'b1);
wire any_pressed   = left_pressed | right_pressed;
```

Ⓑ **Movement speed limiting by `move_tick`.** If we updated the board position at every 25 MHz clock, one long press would drive the board to the boundary almost instantly. Therefore, we introduce a divider counter (`mv_cnt`) and generate a low-rate tick `move_tick`. The board position is updated only when `any_pressed` and `move_tick` are both true:

```
localparam integer DIV      = (CLK_HZ + MOVE_HZ - 1) /
   MOVE_HZ;
localparam integer DIV_W    = $clog2(DIV);
reg [DIV_W-1:0] mv_cnt;
```

```
wire move_tick = (mv_cnt == DIV-1);

always @(posedge clk or negedge sys_rst_n) begin
    if (!sys_rst_n)                        mv_cnt <= {DIV_W
       {1'b0}};
    else if (!any_pressed)                 mv_cnt <= {DIV_W
       {1'b0}};
    else if (move_tick)                    mv_cnt <= {DIV_W
       {1'b0}};
    else                                   mv_cnt <= mv_cnt
        + 1'b1;
end
```

Ⓒ **Candidate update + boundary clamp.** We first compute a candidate next position (move left/right by `STEP`), and then clamp it to [`BOUNDARY_L`, `BOUNDARY_R`]. This ensures the board never moves out of screen:

```
wire [10:0] cand_w = right_pressed ? next_right :
                     left_pressed  ? next_left  : bx_w;

wire [9:0]  next_clamped =
    (cand_w < L_w) ? BOUNDARY_L :
    (cand_w > R_w) ? BOUNDARY_R :
                     cand_w[9:0];

always @(posedge clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        board_x <= 10'd320;
    end else if (any_pressed && move_tick) begin
        board_x <= next_clamped;
    end
end
```

(3) Rendering: `vga_pic_move_board`. This module draws the board as a rectangle centered at `x_board` and with a fixed `BOARD_Y`. At the code level, the rendering is done by:

Ⓐ computing rectangle boundaries `left, right, top, bottom` (with saturation/clamp),

Ⓑ checking the membership condition `in_board`,

Ⓒ selecting output color by a simple conditional assignment.

The pixel-level membership test is:

```
wire in_board = (pix_x >= left)  && (pix_x <= right) &&
                (pix_y >= top)   && (pix_y <= bottom);

always @(*) begin
    pix_data = in_board ? BOARD_COLOR : BACKGROUND_COLOR;
end
```

Integration module (using the three submodules). In the wrapper module, we instantiate: `pll` to generate `vga_clk`, `boardMoveCtl` to output `x_board`, `vga_pic_move_board` to generate `pix_data_board`, and `vga_ctrl` to generate `hsync`, `vsync`, and `rgb`. This structure makes the board controller reusable as a clean functional block in later integration (ball + bricks + FSM pages).

Full code listings . For completeness and consistency with our submission, we include the following full Verilog files *without any modification*.

```
module pressButton(// modified from the instructor lecture
   material
    input wire vga_clk,
    input wire sys_rst_n,
    input wire up, // the button
    output wire key_up // debounce and press effect (
       continuous movement)
);
// Debounce
parameter CNT_20MS_MAX = 20'd500000;
reg key_up_flag;
// counter for press
reg [0:19] cnt_press_20ms;
always @(posedge vga_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        cnt_press_20ms <= 20'b0;
    end else if (up) begin
        cnt_press_20ms <= 20'b0;
    end else if (cnt_press_20ms == CNT_20MS_MAX && !up) begin
```

```
18          cnt_press_20ms <= cnt_press_20ms;
19      end else begin
20          cnt_press_20ms <= cnt_press_20ms + 1'b1;
21      end
22  end
23  // counter for release
24  reg [0:19] cnt_release_20ms;
25  always @(posedge vga_clk or negedge sys_rst_n) begin
26      if (!sys_rst_n) begin
27          cnt_release_20ms <= 20'b0;
28      end else if (!up) begin
29          cnt_release_20ms <= 20'b0;
30      end else if (cnt_release_20ms == CNT_20MS_MAX && up)
           begin
31          cnt_release_20ms <= cnt_release_20ms;
32      end else begin
33          cnt_release_20ms <= cnt_release_20ms + 1'b1;
34      end
35  end
36  // update key_up_flag
37  always @(posedge vga_clk or negedge sys_rst_n) begin
38      if (!sys_rst_n) begin
39          key_up_flag <= 1'b1;
40      end else if (cnt_press_20ms >= CNT_20MS_MAX - 1'd1) begin
41          key_up_flag <= 1'b0;
42      end else if (cnt_release_20ms >= CNT_20MS_MAX - 1'd1)
           begin
43          key_up_flag <= 1'b1;
44      end else begin
45          key_up_flag <= key_up_flag;
46      end
47  end
48
49  assign key_up = key_up_flag;
50
51  endmodule
```

Listing 1.3: pressButton.v

```
1   module boardMoveCtl #(
2       parameter integer CLK_HZ      = 25_000_000,
```

```
3       parameter integer MOVE_HZ    = 200,       //
4       parameter [9:0]   STEP       = 10'd5,     //
5       parameter [9:0]   BOUNDARY_L = 10'd40,
6       parameter [9:0]   BOUNDARY_R = 10'd600
7   )(
8       input  wire        clk,
9       input  wire        sys_rst_n,   //
10      input  wire        B1,          // active-low
11      input  wire        B2,          // active-low
12      output wire [9:0]  x_board
13  );
14
15      // ========== 1) pressButton ==========
16      wire B1_OK, B2_OK; //        1       =           , 0= active       -low
17      pressButton u_pb1 (.vga_clk(clk), .sys_rst_n(sys_rst_n), .up(B1), .key_up(B1_OK));
18      pressButton u_pb2 (.vga_clk(clk), .sys_rst_n(sys_rst_n), .up(B2), .key_up(B2_OK));
19
20      //                          (0)         (1)
21      wire left_pressed  = (B1_OK == 1'b0) && (B2_OK == 1'b1);
22      wire right_pressed = (B2_OK == 1'b0) && (B1_OK == 1'b1);
23      wire any_pressed   = left_pressed | right_pressed;
24
25      // ========== 2)                          MOVE_HZ ==========Otherwise, the button will make the board to end directly
26      localparam integer DIV      = (CLK_HZ + MOVE_HZ - 1) / MOVE_HZ;  //
27      localparam integer DIV_W    = $clog2(DIV);
28      reg [DIV_W-1:0] mv_cnt;
29      wire move_tick = (mv_cnt == DIV-1);
30
31      always @(posedge clk or negedge sys_rst_n) begin
32          if (!sys_rst_n)                     mv_cnt <= {DIV_W{1'b0}};
```

```
33          else if (!any_pressed)                mv_cnt <= {DIV_W
               {1'b0}}; //
34          else if (move_tick)                   mv_cnt <= {DIV_W
               {1'b0}};
35          else                                  mv_cnt <= mv_cnt
                + 1'b1;
36      end
37
38      // ========== 3)                  + 11          +
           ==========
39      reg [9:0] board_x = 10'd320;
40
41      //        11
42      wire [10:0] bx_w    = {1'b0, board_x};
43      wire [10:0] step_w = {1'b0, STEP};
44
45      wire [10:0] next_right = bx_w + step_w;
46      wire [10:0] next_left  = (bx_w >= step_w) ? (bx_w -
           step_w) : 11'd0;
47
48      //
49      wire [10:0] cand_w = right_pressed ? next_right :
50                           left_pressed  ? next_left  : bx_w;
51
52      //
53      wire [10:0] L_w    = {1'b0, BOUNDARY_L};
54      wire [10:0] R_w    = {1'b0, BOUNDARY_R};
55      wire [9:0]  next_clamped =
56          (cand_w < L_w) ? BOUNDARY_L :
57          (cand_w > R_w) ? BOUNDARY_R :
58                           cand_w[9:0];
59
60      always @(posedge clk or negedge sys_rst_n) begin
61          if (!sys_rst_n) begin
62              board_x <= 10'd320;
63          end else if (any_pressed && move_tick) begin
64              board_x <= next_clamped;
65          end
66      end
67
```

```
68      assign x_board = board_x;
69  endmodule
```

Listing 1.4: boardMoveCtl.v

```
1   `timescale 1ns / 1ps
2   module vga_pic_move_board (
3       input  wire         vga_clk,    // 25
                M           H           z
4       input  wire         sys_rst_n, //

5       input  wire [9:0]   pix_x,
6       input  wire [9:0]   pix_y,
7       input  wire [9:0]   x_board,    //
                                        x 0 ..639
8       output reg  [15:0] pix_data
9   );
10      //
11      localparam [9:0] H_VALID = 10'd640;
12      localparam [9:0] V_VALID = 10'd480;
13
14      //
15      localparam [15:0] BACKGROUND_COLOR = 16'hF81F; //
16      localparam [15:0] BOARD_COLOR      = 16'h0000; //
17
18      //
19      localparam [9:0] HALF_W_BOARD = 10'd40; //      80
20      localparam [9:0] HALF_H_BOARD = 10'd4;  //      8~9
21      localparam [9:0] BOARD_Y      = 10'd450;
22
23      //
24      localparam [9:0] X_MAX = H_VALID - 10'd1; // 639
25      localparam [9:0] Y_MAX = V_VALID - 10'd1; // 479
26
27      // ===                                                  /
                  ===
28      //


29      wire [9:0] left  = (x_board >= HALF_W_BOARD) ? (x_board -
            HALF_W_BOARD) : 10'd0;
```

```
30
31      //                  11
                                                    X_MAX
32      wire [10:0] right_w = {1'b0, x_board} + {1'b0,
           HALF_W_BOARD};
33      wire [9:0]  right   = (right_w[10:0] > {1'b0, X_MAX}) ?
           X_MAX : right_w[9:0];
34
35      //        BOARD_Y
36      wire [9:0] top    = (BOARD_Y >= HALF_H_BOARD) ? (BOARD_Y
           - HALF_H_BOARD) : 10'd0;
37
38      //        11                                  Y_MAX
39      wire [10:0] bottom_w = {1'b0, BOARD_Y} + {1'b0,
           HALF_H_BOARD};
40      wire [9:0]  bottom   = (bottom_w[10:0] > {1'b0, Y_MAX}) ?
            Y_MAX : bottom_w[9:0];
41
42      //
43      wire in_board = (pix_x >= left)  && (pix_x <= right) &&
44                      (pix_y >= top)   && (pix_y <= bottom);
45
46      //
47      always @(*) begin
48          pix_data = in_board ? BOARD_COLOR : BACKGROUND_COLOR;
49      end
50  endmodule
```

Listing 1.5: vga_pic_move_board.v

```
1   `timescale 1ns/1ps
2
3   module vga_colorbar(
4   input wire sys_clk , //System Clock, 50MHz
5   input wire sys_rst_n , //Reset signal. Low level is effective
6   // input wire [1:0] frame,
7   input wire button_left,
8   input wire button_right,
9   output wire hsync , //Line sync signal
10  output wire vsync , //Field sync signal
11  output wire [15:0] rgb //RGB565 color data
```

```
12
13 );
14
15  ////
16  //\* Parameter and Internal Signal \//
17  ////
18
19  //wire define
20  wire vga_clk ; //VGA working clock, 25MHz
21  wire [9:0] pix_x ; //x coordinate of current pixel
22  wire [9:0] pix_y ; //y coordinate of current pixel
23  wire [15:0] pix_data_board; //color information
24
25
26  ////
27  //\* Instantiation \//
28  ////
29
30  //------------- clk_gen_inst -------------
31  pll pll_inst
32  (
33  .sys_clk(sys_clk),
34  .sys_rst_n(sys_rst_n),
35
36  .vga_clk(vga_clk) // connector1
37  );
38 // ...
39
40 wire signed [9:0] x_board;
41
42 boardMoveCtl u_board_ctl (
43     .clk        (vga_clk),
44     .sys_rst_n  (sys_rst_n),
45     .B1         (button_left),   //
46     .B2         (button_right),  //
47
48     .x_board    (x_board)
49 );
50
51 vga_pic_move_board u_board_draw (
```

```
52      .vga_clk    (vga_clk),
53      .sys_rst_n  (sys_rst_n),
54      .pix_x      (pix_x),
55      .pix_y      (pix_y),
56      .x_board    (x_board),
57
58      .pix_data   (pix_data_board) //              /
                      mux
59  );
60
61  // ...
62
63
64   //-------------- vga_ctrl_inst -------------
65   vga_ctrl vga_ctrl_inst
66   (
67   .vga_clk (vga_clk ), //VGA working clock, 25MHz
68   .sys_rst_n (sys_rst_n ), //Reset signal. Low level is
        effective
69   .pix_data (pix_data_board ), //color information
70
71   .pix_x (pix_x ), //x coordinate of current pixel
72   .pix_y (pix_y ), //y coordinate of current pixel
73   .hsync (hsync ), //Line sync signal
74   .vsync (vsync ), //Field sync signal
75   .rgb (rgb ) //RGB565 color data
76   );
77
78   endmodule
```

Listing 1.6: Top module excerpt (unchanged, other blocks omitted with ...)

(4) game process with single brick

Github:

`Reproduce_ExampleGame/Part3_VideoWithButtons/singleBrick`

In this part of implementation, we firstly focus on the ball–board interaction, and then combine the key components from (1) to (3) to complete a breakout game with only one brick. Compared with the previous modules, the *only new functional block*

introduced here is the ball–board collision effect module `ball_board_effect`. After that, we reuse: (1) wall reflection baseline, (2) single-brick swept collision logic, and (3) button-controlled board position input `x_board`.

Ball–board effect module: `ball_board_effect`. This module implements the collision between the ball and the *top side* of the board. The design principle stated in Materials & Methods is: *treat the board as a movable brick but keep only the valid top collision.* At code level, the implementation is simplified into a **one-step contact test** and a **vertical reflection**:

Ⓐ **Use next-step candidate position.** Instead of using the current ball center (x_0, y_0), the module directly takes the candidate next position `x_next_s`, `y_next_s` from the main module. This matches the "frame-to-frame update" style used previously.

Ⓑ **Collision condition (top-only).** The collision is triggered only when: (i) the ball is moving downward (`delta_y > 0`), (ii) the ball bottom at next step crosses the board top (`ball_bottom_next >= board_top`), and (iii) the next-step x lies within the board horizontal range [`board_left`, `board_right`]. Therefore, side and bottom collisions are naturally excluded.

Ⓒ **Response: flip Δy and clamp to contact line.** Once collision happens, the module only flips the vertical velocity: $\Delta y \leftarrow -\Delta y$, and moves the ball center to the contact position $y = y_{\text{contact}} =$ `board_top` $-$ `BALL_RADIUS`, to avoid embedding into the board.

In summary, `ball_board_effect` provides three outputs: a corrected next position (`x_modified`, `y_modified`) and a corrected vertical velocity `delta_y_modified`. It can be seen as a small "geometry correction layer" inserted before the wall/brick update rules.

Main combination module: `vga_pic_bbbw`. The main module integrates four effects in a single pipeline: **B**all, **B**oard, **B**rick, and **W**alls. Most internal logic is reused from (1)–(3); the key differences in this module are:

– **Insert a new combination stage.** The module instantiates `ball_board_effect` and feeds it with `x_next_s`, `y_next_s`, `delta_y`, and `x_board`. This stage outputs `x_bb`, `y_bb`, `dy_bb`, which are then used in the "no brick hit" update branch.

– **Update priority: brick hit dominates board/wall corrections.** In the frame update logic, if `brick_alive && hit_w` is true, the module executes the brick collision response and deletes the brick. Otherwise, it uses `x_bb`, `y_bb`, `dy_bb` to continue wall and bottom-boundary handling.

– **Bottom-out policy (single-life in this warmup stage).** When the corrected ball position reaches the bottom boundary, `ball_alive` becomes 0. This is consistent with the later Sub-FSM life control design, where bottom-out triggers a "lose one health point" event.

We show below the *newly introduced* module `ball_board_effect` in full, and only keep the key combination parts of `vga_pic_bbbw` that differ from previous modules.

Full code: `ball_board_effect.`

```
1  `timescale 1ns/1ps
2  module ball_board_effect (
3      //                 /
4      input  wire               vga_clk,
5      input  wire               sys_rst_n,
6
7      //
       x_next_s, y_next_s
8      input  wire signed [11:0] x_next_s,
9      input  wire signed [11:0] y_next_s,
10
11     //                                =                 =
12     input  wire signed [9:0]  delta_y,
13
14     //                x     vga_pic_move2_wallBrickBoard
              0     ..639
15     input  wire [9:0]          x_board,
16
17     //
18     output wire signed [11:0] x_modified,
19     output wire signed [11:0] y_modified,
20     output wire signed [9:0]  delta_y_modified
21 );
```

```
22      // =====            =====
23      localparam signed [9:0] BALL_RADIUS = 10'sd10;
24
25      // =====
           vga_pic_move2_wallBrickBoard        =====
26      localparam [9:0] HALF_W_BOARD = 10'd40; //     80
27      localparam [9:0] HALF_H_BOARD = 10'd4;  //     8
28      localparam [9:0] BOARD_Y      = 10'd450;
29
30      //           top
31      wire [9:0] board_top_u =
32          (BOARD_Y >= HALF_H_BOARD) ? (BOARD_Y - HALF_H_BOARD)
               : 10'd0;
33
34      //                              x_next_s/y_next_s
35      wire signed [11:0] board_top   = $signed({1'b0,
           board_top_u});
36      wire signed [11:0] board_left  = $signed({1'b0, x_board})
            - $signed(HALF_W_BOARD);
37      wire signed [11:0] board_right = $signed({1'b0, x_board})
            + $signed(HALF_W_BOARD);
38
39      //                     y =        y +
40      wire signed [11:0] ball_bottom_next = y_next_s + $signed
           ({2'b00, BALL_RADIUS});
41
42      //                                          y
43      //                              ball_bottom = board_top
               y = board_top - BALL_RADIUS
44      wire signed [11:0] y_contact = board_top - $signed({2'b00
           , BALL_RADIUS});
45
46      //
47      // 1)           delta_y           > 0
48      // 2)
49      // 3) x
50      wire colliding_happen =
51             (delta_y > 0)
52          && (ball_bottom_next >= board_top)
53          && (x_next_s >= board_left)
```

```
54          && (x_next_s <= board_right);
55
56      // =====                                        =====
57      reg signed [11:0] x_r;
58      reg signed [11:0] y_r;
59      reg signed [9:0]  dy_r;
60
61      always @(*) begin
62          //
63          x_r  = x_next_s;
64          y_r  = y_next_s;
65          dy_r = delta_y;
66
67          if (colliding_happen) begin
68              //                      +
69              dy_r = -delta_y;   //
70              y_r  = y_contact;  //
71
72              // x_r                    x_next_s
73              //
                        d   e   l   t   a   _   x        delta_x
74          end
75      end
76
77      assign x_modified        = x_r;
78      assign y_modified        = y_r;
79      assign delta_y_modified  = dy_r;
80
81  endmodule
```

Key differences in `vga_pic_bbbw` (excerpt). The following excerpt shows (i) the instantiation of `ball_board_effect`, and (ii) how its outputs are used in the "no brick hit" update branch.

```
1       // Combination Begins====
           ====
2       wire signed [11:0] x_bb;
3       wire signed [11:0] y_bb;
```

```
    wire signed [9:0]  dy_bb;

    ball_board_effect u_ball_board (
        .vga_clk           (vga_clk),
        .sys_rst_n         (sys_rst_n),
        .x_next_s          (x_next_s),    //

        .y_next_s          (y_next_s),
        .delta_y           (delta_y),     //
        .x_board           (x_board),     //                x
        .x_modified        (x_bb),        //

        .y_modified        (y_bb),
        .delta_y_modified  (dy_bb)        //

    );
    // End Combination 1
```

```
            // ====

               /              ====

            // 1)      ball_board_effect

            ball_x <= (x_bb <= X_MIN) ? X_MIN[9:0] :
                      (x_bb >= X_MAX) ? X_MAX[9:0] :
                                        x_bb[9:0];

            ball_y <= (y_bb <= Y_MIN) ? Y_MIN[9:0] :
                      y_bb[9:0];

            // 2)                                    x_bb

            if (x_bb <= X_MIN || x_bb >= X_MAX)
                delta_x <= -delta_x;

            // 3)
            //
                    d     y     _     b     b
            if (y_bb <= Y_MIN)
```

```
18                  delta_y <= -dy_bb;   //
19              else
20                  delta_y <= dy_bb;    //
21
22              // 4)
23              if (y_bb >= Y_MAX)
24                  ball_alive <= 1'b0;
```

(5) game process with multiple bricks

Github: `Reproduce_ExampleGame/Part3_VideoWithButtons/finalEffects_exampleGame`

In this part of implementation, there are two main tasks. Firstly, we need construct a modular module for a single brick. Based on the module, we can construct multiple bricks scenario. Secondly, we need to manage the multiple bricks through some code level techniques and selecting the minimum for all valid collision possibilities, as stated in Materials and Methods part.

Task 1: Modular swept collision module for a single brick

We encapsulate the swept AABB collision computation of *one brick* into a reusable module `brick_swept_one`. Given the current ball state (x_0, y_0), velocity (v_x, v_y), radius r, and the brick rectangle $[L, R] \times [T, B]$, this module outputs:

- `hit`: whether a valid collision happens within the current frame step;
- `t_enter_fp`: the entering time (fixed-point, range $0 \ldots$ `SCALE`);
- `hit_axis_x`: which axis is responsible for the collision normal (left/right face vs. top/bottom face).

At code level, the key idea is to use the time-interval intersection view: compute `tx_enter/tx_exit` and `ty_enter/ty_exit`, then intersect them to obtain `t_enter` and `t_exit`. A collision is valid iff `t_enter` $\leq$ `t_exit` and `t_enter` $\in [0,$ `SCALE`$]$, while additionally excluding the `t=0` false-hit case by checking `inside0`.

Task 2: Managing multiple bricks by selecting the minimum valid entering time

After computing swept collision candidates for all bricks in parallel, we select the *earliest* valid collision (i.e., the minimum `t_enter_fp` among all `hit==1` bricks). This directly implements the design principle: *select the minimum entering time among all valid brick-ball collision possibilities.*

Parallel collision candidates (generate loop). In `vga_pic_bbbw`, we use a `generate-for` block to instantiate `brick_swept_one` for each of the 16 bricks. Each instance provides a candidate triple:

(`brick_hit[k]`, `brick_t_enter[k]`, `brick_hit_axisx[k]`).

Minimum entering time selection (min-reduction). We then scan all candidates and select the brick with smallest `t_enter`:

```
    //      16
    reg                 hit_any;
    reg   [3:0]         hit_idx;
    reg   signed [15:0] best_t;
    reg                 best_axis_x;

    integer k;
    always @(*) begin
        hit_any      = 1'b0;
        hit_idx      = 4'd0;
        best_t       = 16'sh7FFF;
        best_axis_x  = 1'b0;
        for (k = 0; k < N_BRICKS; k = k + 1) begin
            if (brick_hit[k]) begin
                if (!hit_any || (brick_t_enter[k] < best_t
                   )) begin
                    hit_any      = 1'b1;
                    hit_idx      = k[3:0];
                    best_t       = brick_t_enter[k];
                    best_axis_x  = brick_hit_axisx[k];
                end
            end
        end
```

```
23      end
```

Collision response and brick deletion. If a brick is selected (`hit_any==1`), the system advances the ball to the contact point and flips only one velocity component (according to `best_axis_x`). Finally, the hit brick is removed by clearing its alive bit:

```
1               // ==================        1
                                          ==================
2               if (hit_any && brick_alive[hit_idx]) begin
3                   // ====                                swept
                              +         ====
4                   //            best_t                    8

5                   reg signed [27:0] prodx;
6                   reg signed [27:0] prody;
7                   reg signed [11:0] hit_x;
8                   reg signed [11:0] hit_y;
9
10                  prodx = $signed(vx) * $signed(best_t);
11                  prody = $signed(vy) * $signed(best_t);
12                  hit_x = x0 + (prodx >>> 8);
13                  hit_y = y0 + (prody >>> 8);
14
15                  ball_x <= hit_x[9:0];
16                  ball_y <= hit_y[9:0];
17
18                  if (best_axis_x) begin
19                      //                          x

20                      delta_x <= -delta_x;
21                      ball_x  <= hit_x[9:0] + ((vx >= 0) ?
                           10'sd1 : -10'sd1);
22                  end else begin
23                      //                          y

24                      delta_y <= -delta_y;
25                      ball_y  <= hit_y[9:0] + ((vy >= 0) ?
                           10'sd1 : -10'sd1);
26                  end
```

```
27
28                      brick_alive[hit_idx] <= 1'b0;
```

Therefore, in each frame update, even if the ball crosses multiple bricks geometrically, the system always resolves the earliest physically valid collision first, which prevents the "skipping bricks" phenomenon between neighbouring frames.

Complete Verilog codes

In this subsection, we show the complete source codes for the reusable single-brick swept module and the integrated 16-brick game module.

Module 1: `brick_swept_one`

```
1  `timescale 1ns/1ps
2
3  //              swept AABB
4  module brick_swept_one (
5      input  wire signed [11:0] x0,     // ball_x
6      input  wire signed [11:0] y0,     // ball_y
7      input  wire signed [11:0] vx,     // delta_x
8      input  wire signed [11:0] vy,     // delta_y
9      input  wire signed [11:0] r,      // BALL_RADIUS
10
11     input  wire signed [11:0] L,      //     AABB
12     input  wire signed [11:0] R,
13     input  wire signed [11:0] T,
14     input  wire signed [11:0] B,
15     input  wire               brick_alive,
16
17     output wire               hit,        //
18     output wire signed [15:0] t_enter_fp,//         0        ..
          SCALE
19     output wire               hit_axis_x  // 1:              , 0:

20 );
21     //     32bit                          Verilator WIDTH
22     localparam signed [31:0] SCALE_32    = 32'sd256;       // 8
          bit
23     localparam signed [31:0] INF_POS_32  = 32'sd32767;
24     localparam signed [31:0] INF_NEG_32  = -32'sd32767;
25
26     // sign-extend                  32bit
27     wire signed [31:0] x0_ = x0;
28     wire signed [31:0] y0_ = y0;
29     wire signed [31:0] vx_ = vx;
```

```
30      wire signed [31:0] vy_ = vy;
31      wire signed [31:0] r_  = r;
32      wire signed [31:0] L_  = L;
33      wire signed [31:0] R_  = R;
34      wire signed [31:0] T_  = T;
35      wire signed [31:0] B_  = B;
36
37      //
38      wire signed [31:0] Lp = L_ - r_;
39      wire signed [31:0] Rp = R_ + r_;
40      wire signed [31:0] Tp = T_ - r_;
41      wire signed [31:0] Bp = B_ + r_;
42
43      //                                                    t=0
44      wire inside0 = (x0_ >= Lp) && (x0_ <= Rp) &&
45                     (y0_ >= Tp) && (y0_ <= Bp);
46
47      //              /        32bit
48      wire signed [31:0] num1x = (Lp - x0_) * SCALE_32;
49      wire signed [31:0] num2x = (Rp - x0_) * SCALE_32;
50      wire signed [31:0] num1y = (Tp - y0_) * SCALE_32;
51      wire signed [31:0] num2y = (Bp - y0_) * SCALE_32;
52
53      wire signed [31:0] t1x_fp_32 = (vx_ == 0) ? 32'sd0 : (num1x
            / vx_);
54      wire signed [31:0] t2x_fp_32 = (vx_ == 0) ? 32'sd0 : (num2x
            / vx_);
55      wire signed [31:0] t1y_fp_32 = (vy_ == 0) ? 32'sd0 : (num1y
            / vy_);
56      wire signed [31:0] t2y_fp_32 = (vy_ == 0) ? 32'sd0 : (num2y
            / vy_);
57
58      // x          /
59      wire signed [31:0] tx_enter_32 =
60          (vx_ == 0) ? ((x0_ < Lp || x0_ > Rp) ? INF_POS_32 :
               INF_NEG_32)
61                     : ((t1x_fp_32 < t2x_fp_32) ? t1x_fp_32 :
                          t2x_fp_32);
62
63      wire signed [31:0] tx_exit_32  =
64          (vx_ == 0) ? ((x0_ < Lp || x0_ > Rp) ? INF_NEG_32 :
               INF_POS_32)
65                     : ((t1x_fp_32 < t2x_fp_32) ? t2x_fp_32 :
                          t1x_fp_32);
```

```
66
67      // y          /
68      wire signed [31:0] ty_enter_32 =
69          (vy_ == 0) ? ((y0_ < Tp || y0_ > Bp) ? INF_POS_32 :
                INF_NEG_32)
70                     : ((t1y_fp_32 < t2y_fp_32) ? t1y_fp_32 :
                          t2y_fp_32);
71
72      wire signed [31:0] ty_exit_32  =
73          (vy_ == 0) ? ((y0_ < Tp || y0_ > Bp) ? INF_NEG_32 :
                INF_POS_32)
74                     : ((t1y_fp_32 < t2y_fp_32) ? t2y_fp_32 :
                          t1y_fp_32);
75
76      //             /
77      wire signed [31:0] t_enter_32 =
78          (tx_enter_32 > ty_enter_32) ? tx_enter_32 : ty_enter_32
                ;
79      wire signed [31:0] t_exit_32  =
80          (tx_exit_32  < ty_exit_32)  ? tx_exit_32  : ty_exit_32;
81
82      wire hit_local =
83          brick_alive &&
84          (!inside0) &&
85          (t_enter_32 <= t_exit_32) &&
86          (t_enter_32 >= 32'sd0) &&
87          (t_enter_32 <= SCALE_32);
88
89      assign hit         = hit_local;
90      assign t_enter_fp  = t_enter_32[15:0];           //
                  16
91      assign hit_axis_x  = (tx_enter_32 > ty_enter_32);//
92
93  endmodule
```

Module 2: vga_pic_bbbw **(16 bricks)**

```
1   `timescale 1ns/1ps
2
3   // 16        +     +         + VGA
                                              + crossing
4   module vga_pic_bbbw(
5       input  wire         vga_clk,      // 25 MHz
6       input  wire         sys_rst_n,    // active-low
```

```
7     input  wire [9:0]  pix_x,        // VGA                x
          (0..639)
8     input  wire [9:0]  pix_y,        // VGA                y
          (0..479)
9     input  wire        up,           //
10    input  wire [9:0]  x_board,      //
                  x
11    output reg  [15:0] pix_data      //
12 );
13
14    // =====                              &         =====
15    localparam [9:0]  H_VALID = 10'd640;
16    localparam [9:0]  V_VALID = 10'd480;
17
18    localparam [15:0] BLUE    = 16'h001F;
19    localparam [15:0] PURPPLE = 16'hF81F;
20    localparam [15:0] RED     = 16'hF800;
21
22    localparam [15:0] BALL_COLOR       = BLUE;
23    localparam [15:0] BRICK_COLOR      = 16'hFFFF;
24    localparam [15:0] BACKGROUND_COLOR = PURPPLE;
25    localparam [15:0] BOARD_COLOR      = 16'h0000; //
26
27    // =====              =====
28    reg  signed [9:0] ball_x;
29    reg  signed [9:0] ball_y;
30    localparam signed [9:0] BALL_RADIUS = 10'sd10;
31
32    reg ball_alive;
33
34    //        (signed)
                                                 10'sd20 / 10'sd40
35    reg  signed [9:0] delta_x;
36    reg  signed [9:0] delta_y;
37
38    // =====                                          =====
39    wire frame_tick = (pix_y == V_VALID - 1) && (pix_x ==
          H_VALID - 1);
40
41    // =====                                                 =====
42    wire signed [11:0] x_next_s = $signed({1'b0, ball_x}) +
          $signed(delta_x);
43    wire signed [11:0] y_next_s = $signed({1'b0, ball_y}) +
          $signed(delta_y);
```

```
44
45      // =====
            ball_board_effect =====
46      wire signed [11:0] x_bb;
47      wire signed [11:0] y_bb;
48      wire signed [9:0]  dy_bb;
49
50      ball_board_effect u_ball_board (
51          .vga_clk          (vga_clk),
52          .sys_rst_n        (sys_rst_n),
53          .x_next_s         (x_next_s),   //
54          .y_next_s         (y_next_s),
55          .delta_y          (delta_y),    //
56          .x_board          (x_board),    //              x
57          .x_modified       (x_bb),       //
58          .y_modified       (y_bb),
59          .delta_y_modified (dy_bb)       //
60      );
61
62      // =====     &                                      =====
63      localparam signed [11:0] X_MIN = $signed({1'b0, BALL_RADIUS
            });
64      localparam signed [11:0] X_MAX = $signed({1'b0, H_VALID  -
            BALL_RADIUS});
65      localparam signed [11:0] Y_MIN = $signed({1'b0, BALL_RADIUS
            });
66      localparam signed [11:0] Y_MAX = $signed({1'b0, V_VALID  -
            BALL_RADIUS});
67
68      // ===== 16                  =====
69      localparam integer N_BRICKS     = 16;
70      localparam integer BRICK_ROWS   = 2;
71      localparam integer BRICK_COLS   = 8;
72      localparam [9:0]   BRICK_W      = 10'd40;
73      localparam [9:0]   BRICK_H      = 10'd16;
74      localparam [9:0]   BRICK_X0     = 10'd80;  //
                               x
75      localparam [9:0]   BRICK_Y0     = 10'd80;  //
                               y
76      localparam [9:0]   BRICK_X_GAP  = 10'd4;   //
```

```
77      localparam [9:0]    BRICK_Y_GAP   = 10'd4;    //
78
79      //
80      reg [N_BRICKS-1:0] brick_alive;
81
82      // ----                                          ----
83      function [9:0] brick_L;
84          input [3:0] idx;
85          reg   [2:0] col;
86          begin
87              col      = idx[2:0];
88              brick_L  = BRICK_X0 + col * (BRICK_W + BRICK_X_GAP)
                    ;
89          end
90      endfunction
91
92      function [9:0] brick_T;
93          input [3:0] idx;
94          reg         row;
95          begin
96              row      = idx[3];
97              brick_T  = BRICK_Y0 + row * (BRICK_H + BRICK_Y_GAP)
                    ;
98          end
99      endfunction
100
101     // =====     swept              12bit        =====
102     wire signed [11:0] x0 = ball_x;
103     wire signed [11:0] y0 = ball_y;
104     wire signed [11:0] vx = delta_x;
105     wire signed [11:0] vy = delta_y;
106     wire signed [11:0] r  = BALL_RADIUS;
107
108     // =====                          brick_swept_one =====
109     wire               brick_hit       [0:N_BRICKS-1];
110     wire signed [15:0] brick_t_enter   [0:N_BRICKS-1];
111     wire               brick_hit_axisx [0:N_BRICKS-1];
112
113     genvar gi;
114     generate
115         for (gi = 0; gi < N_BRICKS; gi = gi + 1) begin :
                GEN_BRICK_COLL
116             wire [3:0] idx = gi[3:0];
117             wire [9:0] L0  = brick_L(idx);
```

```
118             wire [9:0] T0  = brick_T(idx);
119             wire [9:0] R0  = L0 + BRICK_W;
120             wire [9:0] B0  = T0 + BRICK_H;
121 
122             //           12bit
123             wire signed [11:0] L_12 = {2'b00, L0};
124             wire signed [11:0] R_12 = {2'b00, R0};
125             wire signed [11:0] T_12 = {2'b00, T0};
126             wire signed [11:0] B_12 = {2'b00, B0};
127 
128             brick_swept_one u_brick_coll (
129                 .x0          (x0),
130                 .y0          (y0),
131                 .vx          (vx),
132                 .vy          (vy),
133                 .r           (r),
134                 .L           (L_12),
135                 .R           (R_12),
136                 .T           (T_12),
137                 .B           (B_12),
138                 .brick_alive (brick_alive[gi]),
139                 .hit         (brick_hit[gi]),
140                 .t_enter_fp  (brick_t_enter[gi]),
141                 .hit_axis_x  (brick_hit_axisx[gi])
142             );
143         end
144     endgenerate
145 
146     //     16
147     reg               hit_any;
148     reg  [3:0]        hit_idx;
149     reg  signed [15:0] best_t;
150     reg               best_axis_x;
151 
152     integer k;
153     always @(*) begin
154         hit_any     = 1'b0;
155         hit_idx     = 4'd0;
156         best_t      = 16'sh7FFF;
157         best_axis_x = 1'b0;
158         for (k = 0; k < N_BRICKS; k = k + 1) begin
159             if (brick_hit[k]) begin
160                 if (!hit_any || (brick_t_enter[k] < best_t))
                    begin
161                     hit_any     = 1'b1;
```

```
162                     hit_idx     = k[3:0];
163                     best_t      = brick_t_enter[k];
164                     best_axis_x = brick_hit_axisx[k];
165                 end
166             end
167         end
168     end
169
170     // =====                 =====
171     localparam [9:0] HALF_W_BOARD = 10'd40;
172     localparam [9:0] HALF_H_BOARD = 10'd4;
173     localparam [9:0] BOARD_Y      = 10'd450;
174
175     localparam [9:0] X_MAX_PIX = H_VALID - 10'd1;
176     localparam [9:0] Y_MAX_PIX = V_VALID - 10'd1;
177
178     wire [9:0] left =
179         (x_board >= HALF_W_BOARD) ? (x_board - HALF_W_BOARD) :
               10'd0;
180
181     wire [10:0] right_w  = {1'b0, x_board} + {1'b0,
           HALF_W_BOARD};
182     wire [9:0]  right    = (right_w[10:0] > {1'b0, X_MAX_PIX})
           ? X_MAX_PIX : right_w[9:0];
183
184     wire [9:0] top =
185         (BOARD_Y >= HALF_H_BOARD) ? (BOARD_Y - HALF_H_BOARD) :
               10'd0;
186
187     wire [10:0] bottom_w = {1'b0, BOARD_Y} + {1'b0,
           HALF_H_BOARD};
188     wire [9:0]  bottom   = (bottom_w[10:0] > {1'b0, Y_MAX_PIX})
           ? Y_MAX_PIX : bottom_w[9:0];
189
190     wire in_board =
191         ($unsigned(pix_x) >= left)  && ($unsigned(pix_x) <=
               right) &&
192         ($unsigned(pix_y) >= top)   && ($unsigned(pix_y) <=
               bottom);
193
194     // =====                         =====
195     wire signed [11:0] dx =
196         $signed({1'b0, pix_x}) - $signed({1'b0, ball_x});
197     wire signed [11:0] dy =
198         $signed({1'b0, pix_y}) - $signed({1'b0, ball_y});
```

```
199
200     wire [23:0] dx2   = dx * dx;
201     wire [23:0] dy2   = dy * dy;
202     wire [24:0] dist2 = dx2 + dy2;
203     localparam [24:0] R2 = 25'd10 * 25'd10;
204
205     wire in_ball = (dist2 <= R2) && ball_alive;
206
207     // ===== 16                  =====
208     wire [N_BRICKS-1:0] in_brick_vec;
209
210     generate
211         for (gi = 0; gi < N_BRICKS; gi = gi + 1) begin :
                GEN_BRICK_RASTER
212             wire [3:0] idx   = gi[3:0];
213             wire [9:0] L0    = brick_L(idx);
214             wire [9:0] T0    = brick_T(idx);
215             wire [9:0] R0    = L0 + BRICK_W;
216             wire [9:0] B0    = T0 + BRICK_H;
217
218             assign in_brick_vec[gi] =
219                 brick_alive[gi] &&
220                 ($signed(pix_x) >= $signed(L0)) && ($signed(
                        pix_x) < $signed(R0)) &&
221                 ($signed(pix_y) >= $signed(T0)) && ($signed(
                        pix_y) < $signed(B0));
222         end
223     endgenerate
224
225     wire in_bricks = |in_brick_vec;
226
227     // =====                  reset           =====
228     initial begin
229         ball_x      = 10'sd320;
230         ball_y      = 10'sd240;
231         delta_x     = 10'sd40;
232         delta_y     = 10'sd40;
233         ball_alive  = 1'b1;
234         brick_alive = {N_BRICKS{1'b1}};
235     end
236
237     // =====         16             + crossing        &
                =====
238     always @(posedge vga_clk or negedge sys_rst_n) begin
239         if (!sys_rst_n) begin
```

```
240             ball_x      <= 10'sd320;
241             ball_y      <= 10'sd240;
242             delta_x     <= 10'sd40;
243             delta_y     <= 10'sd40;
244             ball_alive  <= 1'b1;
245             brick_alive <= {N_BRICKS{1'b1}};
246         end else if (frame_tick) begin
247             // ==================        1
                ==================
248             if (hit_any && brick_alive[hit_idx]) begin
249                 // ====                            swept         +
                           ====
250                 //           best_t                   8

251                 reg signed [27:0] prodx;
252                 reg signed [27:0] prody;
253                 reg signed [11:0] hit_x;
254                 reg signed [11:0] hit_y;
255
256                 prodx = $signed(vx) * $signed(best_t);
257                 prody = $signed(vy) * $signed(best_t);
258                 hit_x = x0 + (prodx >>> 8);
259                 hit_y = y0 + (prody >>> 8);
260
261                 ball_x <= hit_x[9:0];
262                 ball_y <= hit_y[9:0];
263
264                 if (best_axis_x) begin
265                     //                        x

266                     delta_x <= -delta_x;
267                     ball_x  <= hit_x[9:0] + ((vx >= 0) ? 10'sd1
                            : -10'sd1);
268                 end else begin
269                     //                        y

270                     delta_y <= -delta_y;
271                     ball_y  <= hit_y[9:0] + ((vy >= 0) ? 10'sd1
                            : -10'sd1);
272                 end
273
274                 brick_alive[hit_idx] <= 1'b0;
275
276             end else begin
```

```
277                 // ==================        2
                       ==================
278                 //              next           crossing
279                 reg signed [9:0] nx, ny;
280                 reg signed [9:0] ndx, ndy;
281
282                 // 0)
283                 nx  = x_bb[9:0];
284                 ny  = y_bb[9:0];
285                 ndx = delta_x;
286                 ndy = dy_bb;
287
288                 // ---------- X                    crossing
                       ----------
289                 if ( (delta_x < 0) &&                    //
290                      (ball_x  > X_MIN[9:0]) &&           //
291                      (x_bb    <= X_MIN) ) begin          //
292                     nx  = X_MIN[9:0];
293                     ndx = -delta_x;
294                 end else if ( (delta_x > 0) &&           //
295                               (ball_x  < X_MAX[9:0]) &&  //
296                               (x_bb    >= X_MAX) ) begin //
297                     nx  = X_MAX[9:0];
298                     ndx = -delta_x;
299                 end else begin
300                     //          c r o s s i n g
301                     if (x_bb <= X_MIN)
302                         nx = X_MIN[9:0];
303                     else if (x_bb >= X_MAX)
304                         nx = X_MAX[9:0];
305                     else
306                         nx = x_bb[9:0];
307                 end
308
309                 // ---------- Y                  crossing
                       ----------
310                 if ( (ndy < 0) &&                        //
```

```
311                     (ball_y > Y_MIN[9:0]) &&            //
312                     (y_bb   <= Y_MIN) ) begin           //
313                     ny  = Y_MIN[9:0];
314                     ndy = -ndy;                         //
315                 end else begin
316                     //          c r o s s i n g
317                     if (y_bb <= Y_MIN)
318                         ny = Y_MIN[9:0];
319                     else
320                         ny = y_bb[9:0];
321                 end
322
323                 // ----------
                       ----------
324                 if ( (ndy > 0) &&                       //
325                     (ball_y < Y_MAX[9:0]) &&            //
326                     (y_bb   >= Y_MAX) ) begin           //
327                     ny          = Y_MAX[9:0];
328                     ball_alive <= 1'b0;
329                 end else if (ny >= Y_MAX[9:0]) begin
330                     ny          = Y_MAX[9:0];
331                     ball_alive <= 1'b0;
332                 end
333
334                 // ----------              ----------
335                 ball_x  <= nx;
336                 ball_y  <= ny;
337                 delta_x <= ndx;
338                 delta_y <= ndy;
339             end
340         end
341     end
342
343     // =====                    >     >     >        =====
344     always @(*) begin
345         if (!sys_rst_n)
346             pix_data = 16'h0000;
347         else if (in_ball)
348             pix_data = BALL_COLOR;
```

```
349         else if (in_bricks)
350             pix_data = BRICK_COLOR;
351         else if (in_board)
352             pix_data = BOARD_COLOR;
353         else
354             pix_data = BACKGROUND_COLOR;
355     end
356 
357 endmodule
```

1.3.2 For Page Controllers

As the design stated in "Material and Methods", we implement two Moore-type FSMs to manage the whole system. The **Main FSM** (`screen_ctl`) controls the global page transitions (START / MAIN_GAME / CLEAR / GAME_OVER), while the **Sub FSM** (`sub_FSM_ctl`) manages the internal game status (life level, ball reset, and two feedback flags). In implementation, we keep the logic modular: the Sub FSM reports `N_H_P` / `N_B` back to the Main FSM, and the Main FSM generates a one-cycle `new_game_pulse` to (re)initialize the Sub FSM for a new round.

(1) Main Controller: `screen_ctl`

There are two important aspects of the `screen_ctl` module. One is the Moore-type FSM for page switching, and the other is how to detect the falling edge of the raw button signal (idle=1, pressed=0), such that *one press corresponds to exactly one page transition.*

1. Moore FSM (1) State encoding.

```
localparam [2:0] S_START     = 3'd0,
                 S_MAIN_GAME = 3'd1,
                 S_GAME_OVER = 3'd2,
                 S_CLEAR     = 3'd3;
```

(2) State register.

```
always @(posedge clk or negedge rstn) begin
    if (!rstn)
        st_cur <= S_START;
    else
        st_cur <= st_next;
end
```

(3) Next-state generation. The next-state logic strictly follows our design rules: `START` waits for a valid press (`btn_fall`) to enter `MAIN_GAME`; in `MAIN_GAME`, the Sub FSM feedback has higher priority: `N_H_P` leads to `GAME_OVER`, otherwise `N_B` leads to `CLEAR`.

```
always @(*) begin
    st_next = st_cur;
    case (st_cur)
```

```
        S_START: begin
            if (btn_fall)
                st_next = S_MAIN_GAME;
        end
        S_MAIN_GAME: begin
            if (N_H_P == 1'b1)
                st_next = S_GAME_OVER;
            else if (N_B == 1'b1)
                st_next = S_CLEAR;
        end
        default: begin
            st_next = S_START;
        end
    endcase
end
```

(4) Output generation (Moore). Since it is a Moore FSM, the output depends only on the current state. We register `frame` with `st_cur`:

```
always @(posedge clk or negedge rstn) begin
    if (!rstn)
        frame_r <= S_START;
    else
        frame_r <= st_cur;
end
assign frame = frame_r;
```

2. Falling-edge detection and `new_game_pulse` To avoid multiple transitions under a long press, we detect only the **falling edge** (idle=1 to pressed=0) of the raw button. Technically, we store the previous sampled value `button_d` and form a one-cycle pulse:

```
always @(posedge clk or negedge rstn) begin
    if (!rstn)
        button_d <= 1'b1;
    else
        button_d <= button;
end
assign btn_fall = (button_d == 1'b1) && (button == 1'b0);
```

Based on `btn_fall`, we further generate `new_game_pulse` only when the system leaves `START` to begin a new round:

```
if (st_cur == S_START && btn_fall)
    new_game_r <= 1'b1;
```

Therefore, `frame` indicates the current page, and `new_game_pulse` serves as a clean "start signal" to initialize the Sub FSM.

(2) Sub Controller: `sub_FSM_ctl`

There are two important aspects of the `sub_FSM_ctl` module. One is the Moore-type FSM for life-level evolution, and the other is how to extract a valid *ball-lost event* from `ball_alive` using a falling-edge detector, while ensuring it is only active in the `MAIN_GAME` page.

1. Moore FSM for life management (1) State encoding.

```
localparam [2:0] ST_3L    = 3'd0;
localparam [2:0] ST_2L    = 3'd1;
localparam [2:0] ST_1L    = 3'd2;
localparam [2:0] ST_DIE   = 3'd3;
localparam [2:0] ST_CLEAR = 3'd4;
```

(2) Key outputs. The Sub FSM outputs (i) `life_level` (3/2/1/0), (ii) a one-cycle `reset_bb` when losing one life (used to reset ball/board while keeping bricks), and (iii) two flags `N_H_P` / `N_B` to inform the Main FSM about GAME_OVER or CLEAR.

2. Ball-lost event detection (falling edge of `ball_alive`) We only care about ball-lost events inside `MAIN_GAME`. Thus we gate the event with `in_game` and detect `ball_alive:` $1 \rightarrow 0 by storing$`ball_alive_d` :

```
wire in_game       = (frame == FRAME_MAIN_GAME);
wire ball_lost_evt = in_game && (ball_alive_d == 1'b1) && (ball_alive == 1'b0);
```

This guarantees that one ball drop produces exactly one event pulse.

3. State transitions and one-life reset At the beginning of each new round, `new_game` acts as a *local reset*: it sets life to 3, clears flags, and triggers `reset_bb` once:

```
if (new_game) begin
    st_cur     <= ST_3L;
    life_r     <= 2'd3;
    N_H_P_r    <= 1'b0;
    N_B_r      <= 1'b0;
    reset_bb_r <= 1'b1;
end
```

During normal gameplay, the Sub FSM follows the intended logic: (i) if `no_bricks` becomes 1 in any life state, the system enters `ST_CLEAR` and asserts `N_B`; (ii) otherwise, each `ball_lost_evt` decreases life by one and asserts `reset_bb` (except when life reaches 0, where it asserts `N_H_P` and enters `ST_DIE`). For example:

```
ST_3L: if (no_bricks) ... else if (ball_lost_evt) ... -> ST_2L, reset_bb_r=1
ST_2L: if (no_bricks) ... else if (ball_lost_evt) ... -> ST_1L, reset_bb_r=1
ST_1L: if (no_bricks) ... else if (ball_lost_evt) ... -> ST_DIE, N_H_P_r=1
```

After entering `ST_DIE` or `ST_CLEAR`, the corresponding flag remains asserted until the next `new_game` pulse clears it, which matches our top-level page-control strategy.

Summary of the two-FSM cooperation Overall, the Main FSM decides *which page to show* via `frame`, while the Sub FSM decides *how the game evolves internally* via `life_level`, `reset_bb`, and the two completion flags `N_H_P`/`N_B`. This separation keeps the implementation consistent with the design flow: global page scheduling and local game dynamics are decoupled, and they communicate only through a minimal set of clean handshake signals.

1.3.3 For Static Pages

1.3.4 For Dynamic Page

1.3.5 Overall Combination

Github: `Reproduce_ExampleGame/Part5_combine2`
Now, we have completed each small module for the breakout video game system. However, the system is too complex to directly combine in a single step. Therefore, we finally split the implementation into several clear functional blocks, verify them independently, and then integrate them into a complete system that produces the expected VGA video effects.

Source files in this part

The folder `Part5_combine2` contains the following Verilog source files:

- `pll.v`: generate `vga_clk` (25 MHz) from `sys_clk`.
- `vga_ctrl.v`: VGA timing controller; outputs `pix_x`, `pix_y`, `hsync`, `vsync`, `rgb`.
- `pressButton.v`: button debounce (raw button → stable level).
- `boardMoveCtl.v`: board position controller; outputs `x_board`.
- `screen_ctl.v`: *Main* Moore-type FSM for page control; outputs `frame` and `new_game_pulse`.
- `sub_FSM_ctl.v`: *Sub* FSM for life management; outputs `life_level`, `reset_bb`, `N_H_P`, `N_B`.
- `ball_board_effect1.v`: ball–board collision correction (used inside the game engine).
- `brick_swept_one.v`: swept AABB collision for a single brick (reused by multiple bricks).
- `vga_pic_bbbw1.v`: main game engine (ball + board + multiple bricks) and game rendering; outputs `pix_data_game`, `ball_alive_flag`, `no_bricks_flag`.
- `vga_pic_start.v`, `vga_pic_gameover.v`, `vga_pic_clear.v`, `vga_pic_end.v`: static pages.

– `hearts_overlay.v`: heart overlay mask and LED mapping based on `life_level`.

– `vga_scene.v`: page composition (select page by `frame`, and overlay hearts on MAIN_GAME).

– `vga_colorbar.v`: top-level integration module that instantiates and connects all submodules above.

System-level signal flow

At the top level, `vga_colorbar.v` acts as the integration wrapper. The key connections are summarized as follows:

Ⓐ **Clock and pixel scanning.** `pll.v` generates `vga_clk` (25 MHz). Then `vga_ctrl.v` uses `vga_clk` to scan the VGA screen and outputs pixel coordinates `pix_x` and `pix_y`. All rendering modules are driven by `pix_x`/`pix_y`.

Ⓑ **Page controllers (two FSMs).** The Main FSM `screen_ctl.v` outputs the page index `frame` and a one-clock pulse `new_game_pulse` when starting a new round. The Sub FSM `sub_FSM_ctl.v` receives `ball_alive_flag` and `no_bricks_flag` from the game engine, and produces (i) the life counter `life_level`, (ii) the reset pulse `reset_bb` for "lose one life", and (iii) two terminal flags `N_H_P`/`N_B` which feed back to the Main FSM for page switching.

Ⓒ **Board control.** `boardMoveCtl.v` converts button input into the board center position `x_board`. Besides continuous movement, `new_game_pulse` is also used to re-center the board at the beginning of each new round.

Ⓓ **Game engine (dynamic gameplay).** `vga_pic_bbbw1.v` is the core module that renders the MAIN_GAME screen. It receives `pix_x`/`pix_y` for rasterization, `x_board` for board geometry, and `reset_bb`/`new_game_pulse` for state reinitialization. It outputs `pix_data_game` and two status flags: `ball_alive_flag` (ball is still in the screen) and `no_bricks_flag` (all bricks cleared).

Ⓔ **Scene composition and overlay.** `vga_scene.v` selects between static pages and the game page according to `frame`. In MAIN_GAME, it overlays hearts based on the mask output `heart_pixel` from `hearts_overlay.v`. Finally, the composed pixel `pix_data` is sent to `vga_ctrl.v`, which drives `rgb` output.

The complete Verilog implementation for this part is provided in the Appendix, including `hearts_overlay.v`, `vga_scene.v`, and the top-level integration module `vga_colorbar.v`.

1.4 Results

1.4.1 Functional Simulation

Virtual Development Board Based Simulation for Game Process

For each key implementation mentioned in "Implementations" part of the report, we used the virtual development board to verify whether the function of the module is correct or not.

(1) Wall and Ball: `vga_pic_move2`
Github:
`Reproduce_ExampleGame/Part2_Video_Without_Buttons/section1_reflectionByWalls`
See video at *Ball and Walls (no falling out) Effect*:
`https://www.bilibili.com/video/BV1yhBxBrER5/?share_source=copy_web&vd_source=947dff25090943e80326c35433b905d0`
For convenience, we provide three screenshots from the video. From the left figure to the right figure, you can see the ball moves as expected, i.e., it travels in a straight line and is reflected by the wall following the mirror reflection principle.

Fig. 1.25. Wall and ball reflection effect (three screenshots from the video).

(2) Single Brick and Ball
Github:
`Reproduce_ExampleGame/Part2_Video_Without_Buttons/section2_effectsFromBricks/vga_pic_move2.v`
See video at *Ball and Single Brick Effect*:
`https://www.bilibili.com/video/BV118BxBGEYg/?share_source=copy_web&vd_source=947dff25090943e80326c35433b905d0`
For convenience, we provide three screenshots from three neighbouring frames of the video. Note: "Neighbouring" is critical to determine the correctness of this module. In the left figure, the ball collides with the white brick. In the middle figure, the ball is pushed back to the contact position at the bottom side of the brick. Finally, in the right figure, the ball is reflected away from the collision position.

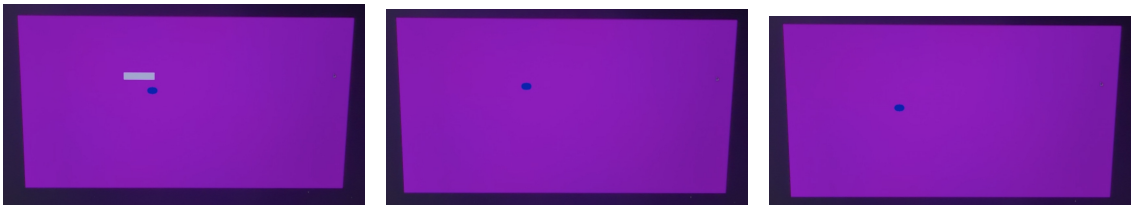


Fig. 1.26. Ball and single brick effect (three neighbouring frames).

(3) Button-Controlled Board Constrained by Wall

Github:

`Reproduce_ExampleGame/Part3_VideoWithButtons/warmup_board`

In this part of verification, we check three functions according to the three designed modules mentioned in "Implementations" part of the report.

See video at *Button-Controlled Board Constrained by Walls*:

`https://www.bilibili.com/video/BV118BxBGExi/?share_source=copy_web&vd_source=947dff25090943e80326c35433b905d0`

We need to check the debounce effect, the movement effect of the board, and the rendering effect. For convenience, we provide five screenshots from the video. In the first row, we tried to make the board move to the left and then go back. In the second row (taken after the right figure of the first row), we tried to move the board further to the right.

Fig. 1.27. Button-controlled board: movement and rendering verification (five screenshots).

From the above screenshots, we can conclude that the movement effect and the rendering effect of the board work correctly. For the debounce effect, please refer to the video. At the beginning of the video, you can maximize the audio volume

and listen carefully. After the initial loud reset button press, there will be a dull and continuous button sound (holding the left button). Meanwhile, the board does not move immediately and shows a noticeable delay, which indicates the success of our debounce design.

(4) Game Process with Single Brick

Github:

`Reproduce_ExampleGame/Part3_VideoWithButtons/singleBrick`

In this part of verification, we check the game process with a single brick.

See video at *Game Process with Single Brick*:

`https://www.bilibili.com/video/BV1peBxBzEfC/?share_source=copy_web&vd_source=947dff25090943e80326c35433b905d0`

Compared to the former three verifications, the critical point here is the ball-board reflection. Therefore, we provide three screenshots taken from three neighbouring frames. Additionally, we provide two screenshots showing the ball falling out of the screen.

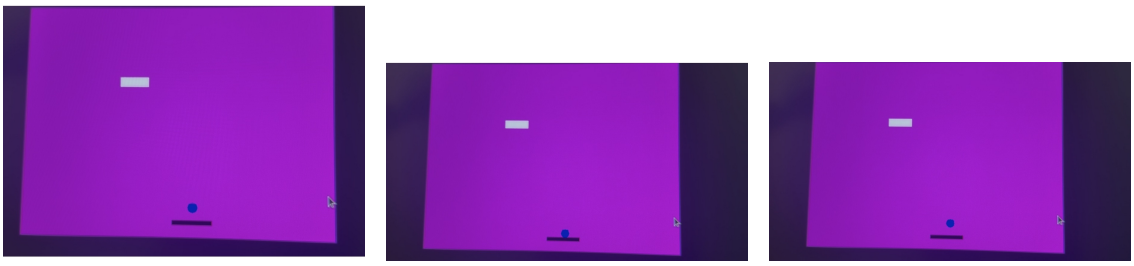


Fig. 1.28. Ball-board reflection (three neighbouring frames).

From left to right, we can see that the ball is first pushed to the top side of the board, and then reflected away from the board, following the mirror reflection principle.

Fig. 1.29. Ball falling-out effect (two screenshots).

From the above, we can see that after the ball falls out of the screen, it disappears as expected in our design.

(5) Game Process with Multiple Bricks

Github:

`Reproduce_ExampleGame/Part3_VideoWithButtons/finalEffects_exampleGame`

In this part, we check the game process with multiple bricks.

See video at *Game Process with Multiple Bricks*:

`https://www.bilibili.com/video/BV1CYBxBgEUo/?share_source=copy_web&vd_source=947dff25090943e80326c35433b905d0`

Compared with the single brick process, the significant point here is the brick cancellation and the correct ball reflection from only one brick for each collision. For convenience, we provide six screenshots taken from two sets of three neighbouring frames.

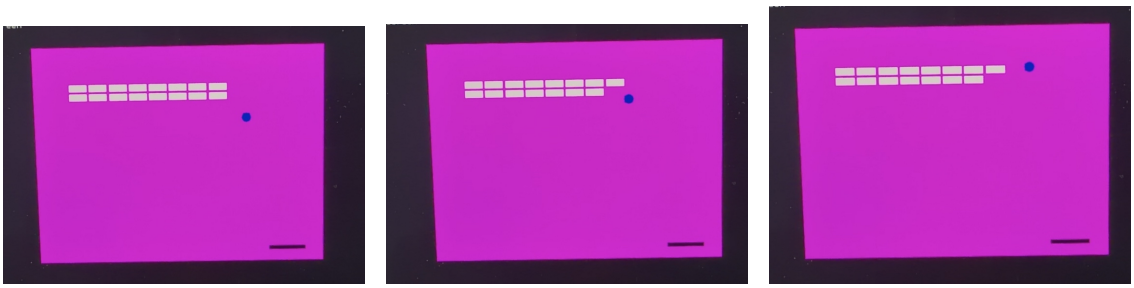


Fig. 1.30. Multiple bricks: the ball hits the right side of a brick (three neighbouring frames).

From left to right, we can see the ball is reflected by the right side of the brick, and the brick is cancelled when the collision happens.

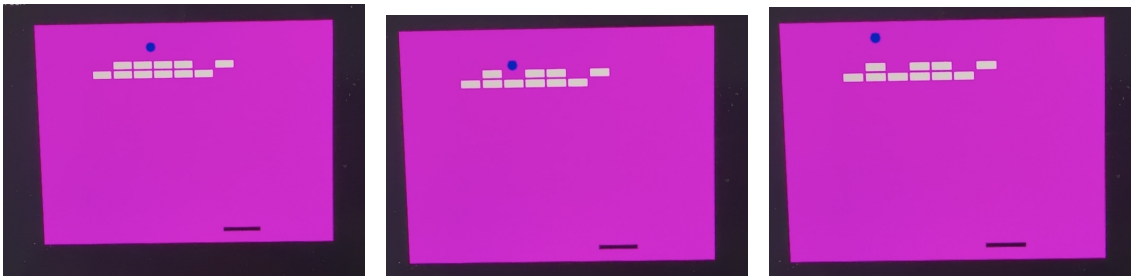


Fig. 1.31. Multiple bricks: the ball hits the top side of a brick (three neighbouring frames).

From left to right, we can see the ball is reflected by the top side of the brick, and the brick is cancelled when the collision happens. Additionally, multiple bricks have already been cancelled, which matches the expected game process.

Functional Simulation for Page Controllers

Notice: In simulation, because of the errors from the Quartus, we choose another online verification tool called "EDA playground" to fulfill the requirements of the functional simulation. The website is `https://www.edaplayground.com/`.

Testbench Source Codes. The full source codes of the two testbenches (`screen_ctl_tb` and `sub_FSM_ctl_tb`) are attached at the end of this subsection " Functional Simulation for Page Controllers" for reference.

Testbench Verification for `screen_ctl` To ensure the correctness of the top-level page-control FSM `screen_ctl`, we wrote a testbench `screen_ctl_tb` and verified its behavior in simulation using waveform and (`$display`/`$fatal`). The verification focuses on the following four functional requirements.

Ⓐ **START → MAIN_GAME triggers a one-cycle `new_game_pulse`.**

How we verified: In the START page, we generated a button falling edge (idle 1 to pressed 0) by driving `button` low before a clock rising edge. Then we checked that `new_game_pulse` becomes `1` for exactly one clock cycle, and returns to `0` in the next cycle.

Ⓑ **MAIN_GAME → CLEAR when `N_B` is asserted.**

How we verified: While staying in MAIN_GAME, we asserted `N_B=1` for one clock cycle and then de-asserted it. We observed that the FSM transitions to the CLEAR page, i.e., `frame` becomes `S_CLEAR`, indicated by "3" in waveform.

Ⓒ **MAIN_GAME → GAME_OVER when `N_H_P` is asserted.**

How we verified: While staying in MAIN_GAME, we asserted `N_H_P=1` for one clock cycle and then de-asserted it. We observed that the FSM transitions to the GAME_OVER page, i.e., `frame` becomes `S_GAME_OVER`, indicated by "2" in waveform.

Ⓓ **Asynchronous reset always returns the system to START.**

How we verified: We toggled the active-low reset `rstn` at a time that is intentionally *not* aligned with the clock edge. The waveform shows that `frame` is forced to `S_START` (indicated by number "0" in waveform) immediately after `rstn=0`, confirming the asynchronous reset behavior. After releasing reset (`rstn=1`), the system remains in START.

From the second figure (generated by Python with some handwritten annotations), we can see that the above four verifications are successful.

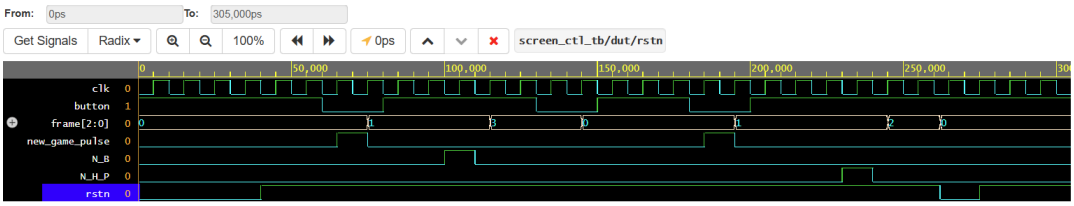


Fig. 1.32. Original simulation waveform of `screen_ctl` (VCD viewer).

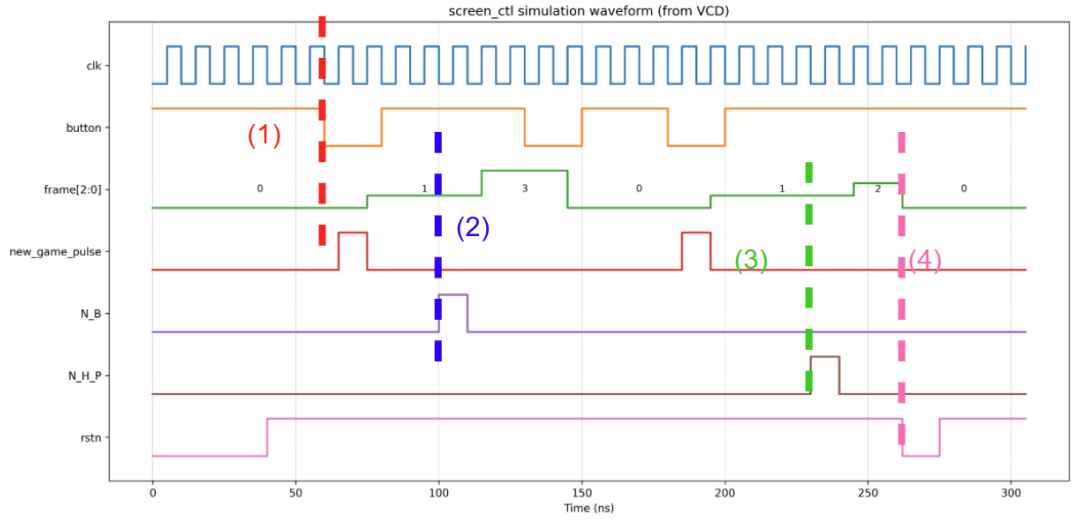


Fig. 1.33. Waveform re-plotted by Python with handwritten annotations highlighting the four checks.

Testbench Verification for `sub_FSM_ctl` To verify the correctness of the sub-FSM `sub_FSM_ctl` (life management, ball loss reset, and CLEAR/GAME_OVER flags), we wrote testbench `sub_FSM_ctl_tb`. The simulation `sub_FSM_ctl_tb.vcd` and print automatic `[PASS]/[FAIL]` messages using `$display` and `$fatal`. The verification includes the following three functional requirements.

Ⓐ **`new_game` initializes the sub-FSM and triggers `reset_bb`.**

What to check: when a new round starts, the life counter should be 3, and the ball/board should be reset once.

How we verified: the testbench generates a one-cycle pulse on `new_game`. Then it checks that `reset_bb` becomes `1` for one clock cycle and returns to `0`, and that `life_level` is restored to `3` afterwards. (In the testbench, the checks are performed at the clock *negedge* to ensure all nonblocking assignments triggered at the previous *posedge* have already settled.)

Ⓑ **`no_bricks` asserts `N_B` (CLEAR) and latches it until `new_game`.**

What to check: once all bricks are cleared, the module should raise the CLEAR flag and keep it asserted.

How we verified: while setting `frame=MAIN_GAME`, the testbench asserts `no_bricks=1` for one cycle. We confirmed that `N_B` is set to `1` and remains latched. Next, we pulse `new_game` again and confirm `N_B` is cleared back to `0`.

Ⓒ **After three ball-loss events, `N_H_P` (GAME_OVER) is asserted and latched.**

What to check: each time the ball falls out, the life counter decreases (3→2→1→0). When life reaches 0, the GAME_OVER flag should be asserted.

How we verified: the testbench repeatedly generates the required ball-loss pattern `ball_alive: 1→0` under `frame=MAIN_GAME`, which triggers the internal ball-lost event detection. After the first, second, and third events, we check that `life_level` becomes 2, 1, and 0, respectively. Finally, we verify that `N_H_P` is asserted (`1`) and stays latched until the next `new_game`.

Simulation Evidence. The simulation console output shows three test cases with `[PASS]` logs (new game reset, no bricks, and life-loss sequence), and the dumped waveform `sub_FSM_ctl_tb.vcd` provides visual confirmation of signal transitions. You can see the two figures below serving as simulation waveforms: the first one is from EDA Playground, and the second one is re-plotted using Python.

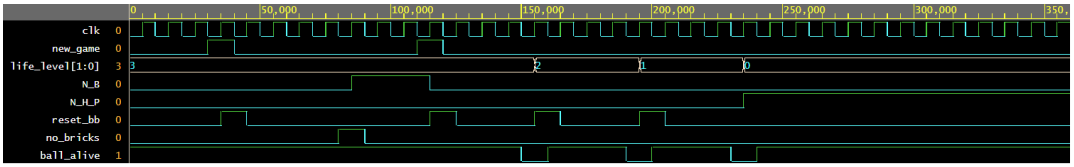


Fig. 1.34. Original simulation waveform of `sub_FSM_ctl` (EDA Playground).

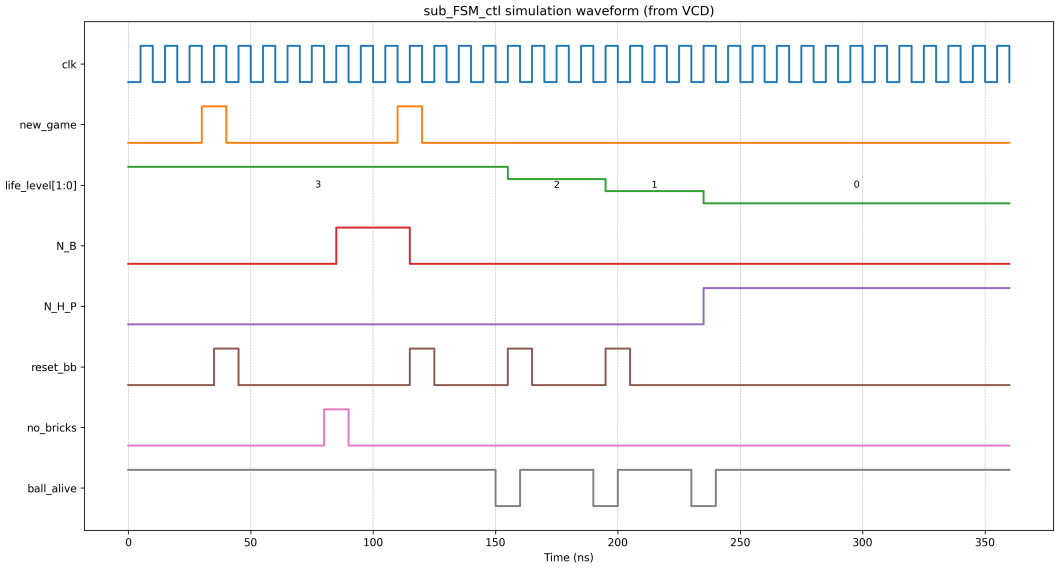


Fig. 1.35. Waveform re-plotted by Python

Testbench Source Codes. For completeness, the full source codes of the two testbenches (`screen_ctl_tb` and `sub_FSM_ctl_tb`) are attached at the end of this subsection for reference.

Appendix: Full Testbench Codes

```
1  `timescale 1ns/1ps
2
3  module screen_ctl_tb;
4
5      // ======================
6      // DUT IO
7      // ======================
8      reg clk;
9      reg rstn; // active-low async reset
10     reg button; // idle=1, pressed=0
11     reg N_H_P;
12     reg N_B;
13     wire [2:0] frame;
```

```
14      wire new_game_pulse;
15
16      // ======================
17      // Instantiate DUT
18      // ======================
19      screen_ctl dut (
20          .clk(clk),
21          .rstn(rstn),
22          .button(button),
23          .N_H_P(N_H_P),
24          .N_B(N_B),
25          .frame(frame),
26          .new_game_pulse(new_game_pulse)
27      );
28
29      // ======================
30      // State encoding (must match DUT)
31      // ======================
32      localparam [2:0] S_START = 3'd0,
33                       S_MAIN_GAME = 3'd1,
34                       S_GAME_OVER = 3'd2,
35                       S_CLEAR = 3'd3;
36
37      // ======================
38      // Clock: 100MHz (10ns period)
39      // ======================
40      initial begin
41          clk = 1'b0;
42          forever #5 clk = ~clk;
43      end
44
45      // ======================
46      // VCD dump for waveform
47      // ======================
48      initial begin
49          $dumpfile("screen_ctl_tb.vcd");
50          $dumpvars(0, screen_ctl_tb);
51      end
52
53      // ======================
```

```
54      // Small helpers / tasks
55      // ======================
56      task wait_clks(input integer n);
57          integer i;
58          begin
59              for (i = 0; i < n; i = i + 1) begin
60                  @(posedge clk);
61              end
62          end
63      endtask
64
65      // Press button (active-low) so that a falling edge is seen at
            next posedge
66      task press_button_one_pulse;
67          begin
68              // ensure idle high
69              button <= 1'b1;
70              @(negedge clk);
71              // drive low before posedge => falling edge detected at
                    upcoming posedge
72              button <= 1'b0;
73              // keep low for 2 cycles (safe)
74              wait_clks(2);
75              @(negedge clk);
76              button <= 1'b1;
77              wait_clks(1);
78          end
79      endtask
80
81      task expect_frame(input [2:0] exp, input [1023:0] tag);
82          begin
83              if (frame !== exp) begin
84                  $display("[FAIL] %s : frame=%0d exp=%0d @%0t", tag,
                        frame, exp, $time);
85                  $fatal;
86              end else begin
87                  $display("[PASS] %s : frame=%0d @%0t", tag, frame,
                        $time);
88              end
89          end
```

```
90      endtask
91
92      task expect_pulse_1cycle(input [1023:0] tag);
93      begin
94          @(posedge clk); #1; //  NBA
95          if (new_game_pulse !== 1'b1) begin
96              $display("[FAIL] %s : new_game_pulse not 1 @%0t", tag,
                    $time);
97              $fatal;
98          end
99          @(posedge clk); #1;
100         if (new_game_pulse !== 1'b0) begin
101             $display("[FAIL] %s : new_game_pulse not back to 0 @%0t",
                    tag, $time);
102             $fatal;
103         end
104         $display("[PASS] %s : new_game_pulse is 1-cycle @%0t", tag,
                $time);
105     end
106 endtask
107
108     // ======================
109     // Main test sequence
110     // ======================
111     initial begin
112         // init
113         rstn = 1'b0;
114         button = 1'b1;
115         N_H_P = 1'b0;
116         N_B = 1'b0;
117
118         // -------- Test 4: reset -> START --------
119         // hold reset low for a while, then release
120         #17; // deliberately not aligned to clk edge
121         rstn = 1'b0;
122         #23;
123         rstn = 1'b1;
124
125         // Because frame_r is reset to START async, frame should be
                START after reset.
```

```
126         // Wait 1-2 clocks for everything to stabilize
127         wait_clks(2);
128         expect_frame(S_START, "After reset released, should be START")
                ;
129
130         // -------- Test 1: START -> MAIN_GAME and new_game_pulse
                asserted --------
131         //  button posedge  pulse
132         button <= 1'b1;
133         @(negedge clk);
134         button <= 1'b0; //  posedge
135         expect_pulse_1cycle("START->MAIN: new_game_pulse");
136         @(negedge clk);
137         button <= 1'b1;
138         wait_clks(2);
139         expect_frame(S_MAIN_GAME, "Should be MAIN_GAME after button
                press");
140
141         // -------- Test 2: MAIN_GAME + N_B -> CLEAR --------
142         // assert N_B for 1 cycle
143         @(negedge clk);
144         N_B <= 1'b1;
145         wait_clks(1);
146         @(negedge clk);
147         N_B <= 1'b0;
148
149         // allow transition + frame lag
150         wait_clks(2);
151         expect_frame(S_CLEAR, "N_B=1 should move MAIN_GAME->CLEAR");
152
153         // return to START via button (CLEAR -> START)
154         press_button_one_pulse();
155         wait_clks(2);
156         expect_frame(S_START, "CLEAR->START by button");
157
158         // -------- Test 3: MAIN_GAME + N_H_P -> GAME_OVER --------
159         // start new game again (button on START)
160         fork
161             begin
162                 press_button_one_pulse();
```

```
163             end
164             begin
165                 expect_pulse_1cycle("START->MAIN (again):
                        new_game_pulse");
166             end
167         join
168         wait_clks(2);
169         expect_frame(S_MAIN_GAME, "Should be MAIN_GAME again");
170 
171         // assert N_H_P for 1 cycle
172         @(negedge clk);
173         N_H_P <= 1'b1;
174         wait_clks(1);
175         @(negedge clk);
176         N_H_P <= 1'b0;
177 
178         wait_clks(2);
179         expect_frame(S_GAME_OVER, "N_H_P=1 should move MAIN_GAME->
                GAME_OVER");
180 
181         // -------- Extra: async reset in the middle (strong waveform
                evidence) --------
182         // While not in START, drop rstn asynchronously and see
                immediate START
183         #7; // not aligned to clk
184         rstn = 1'b0;
185         #13;
186         // frame should already be START due to async reset (no need
                to wait clk)
187         if (frame !== S_START) begin
188             $display("[FAIL] Async reset: frame not START immediately
                    @%0t", $time);
189             $fatal;
190         end else begin
191             $display("[PASS] Async reset forces frame to START
                    immediately @%0t", $time);
192         end
193         rstn = 1'b1;
194         wait_clks(2);
```

```
195         expect_frame(S_START, "After async reset release, stay START")
                ;
196
197         $display("======== ALL TESTS PASSED ========");
198         #20;
199         $finish;
200     end
201
202 endmodule
```

Listing 1.7: Testbench for `screen_ctl` (`screen_ctl_tb`)

```
1  `timescale 1ns/1ps
2
3  module sub_FSM_ctl_tb;
4
5      // --- Signals ---
6      reg clk;
7      reg rstn;
8      reg new_game;
9      reg [2:0] frame;
10     reg ball_alive;
11     reg no_bricks;
12
13     wire N_H_P;
14     wire N_B;
15     wire reset_bb;
16     wire [1:0] life_level;
17
18     // --- DUT Instantiation ---
19     sub_FSM_ctl dut (
20         .clk(clk), .rstn(rstn), .new_game(new_game), .frame(frame),
21         .ball_alive(ball_alive), .no_bricks(no_bricks),
22         .N_H_P(N_H_P), .N_B(N_B), .reset_bb(reset_bb), .life_level(
               life_level)
23     );
24
25     localparam [2:0] FRAME_MAIN_GAME = 3'd1;
26
27     // --- Clock Generation ---
28     initial begin
```

```
29          clk = 1'b0;
30          forever #5 clk = ~clk; // 100MHz
31      end
32
33      // --- VCD Dump ---
34      initial begin
35          $dumpfile("sub_FSM_ctl_tb.vcd");
36          $dumpvars(0, sub_FSM_ctl_tb);
37      end
38
39      // --- Tasks ---
40      task wait_clks(input integer n);
41          integer i;
42          begin
43              for (i = 0; i < n; i = i + 1)
44                  @(posedge clk);
45          end
46      endtask
47
48      task pulse_new_game;
49          begin
50              @(negedge clk);
51              new_game = 1'b1;
52              @(negedge clk); //
53              new_game = 1'b0;
54          end
55      endtask
56
57      task trigger_ball_lost_evt;
58          begin
59              frame = FRAME_MAIN_GAME;
60              @(negedge clk);
61              ball_alive = 1'b1;
62              @(negedge clk);
63              ball_alive = 1'b0; //
64              @(negedge clk);
65              ball_alive = 1'b1; //
66          end
67      endtask
68
```

```
69      //  Task
70      task check_expect(input [31:0] actual, input [31:0] expected,
            input [1023:0] msg);
71          begin
72              @(negedge clk); //  NBA
73              if (actual !== expected) begin
74                  $display("[FAIL] %s | Expected: %0d, Got: %0d @%0t",
                        msg, expected, actual, $time);
75                  $fatal;
76              end else begin
77                  $display("[PASS] %s @%0t", msg, $time);
78              end
79          end
80      endtask
81
82      // --- Test Sequence ---
83      initial begin
84          // Initialization
85          rstn = 1'b0;
86          new_game = 1'b0;
87          frame = 3'd0;
88          ball_alive = 1'b1;
89          no_bricks = 1'b0;
90
91          #13 rstn = 1'b1;
92          wait_clks(2);
93
94          // Point 1: Test Reset and New Game
95          $display("--- Test Case 1: New Game Reset ---");
96          pulse_new_game();
97          //  reset_bb  new_game  posedge  1
98          //
99          check_expect(reset_bb, 1'b1, "new_game -> reset_bb active");
100         check_expect(reset_bb, 1'b0, "reset_bb back to 0");
101         check_expect(life_level, 2'd3, "Initial life_level=3");
102
103         // Point 2: Clear Bricks
104         $display("--- Test Case 2: No Bricks ---");
105         frame = FRAME_MAIN_GAME;
106         @(negedge clk); no_bricks = 1'b1;
```

```
107         @(negedge clk); no_bricks = 1'b0;
108         check_expect(N_B, 1'b1, "no_bricks -> N_B latched");
109
110         pulse_new_game(); // Reset for next test
111         check_expect(N_B, 1'b0, "new_game clears N_B");
112
113         // Point 3: Life Loss Sequence
114         $display("--- Test Case 3: Life Loss Sequence ---");
115         frame = FRAME_MAIN_GAME;
116
117         trigger_ball_lost_evt();
118         check_expect(life_level, 2'd2, "1st ball lost");
119
120         trigger_ball_lost_evt();
121         check_expect(life_level, 2'd1, "2nd ball lost");
122
123         trigger_ball_lost_evt();
124         check_expect(life_level, 2'd0, "3rd ball lost");
125         check_expect(N_H_P, 1'b1, "Game Over (N_H_P) latched");
126
127         $display("================================================");
128         $display(" ALL sub_FSM_ctl TESTS PASSED SUCCESSFULLY ");
129         $display("================================================");
130         #100;
131         $finish;
132     end
133
134 endmodule
```

Listing 1.8: Testbench for `sub_FSM_ctl` (`sub_FSM_ctl_tb`)

1.4.2 Test

We tested the video game along two typical gameplay paths:

Ⓐ from the START page to the GAME OVER page;

Ⓑ from the START page to the CLEAR page.

Since the original recordings are relatively long, we provide accelerated versions. For Path (2), we additionally provide both the original-speed video and the 4×-speed video for clearer observation of the final brick-elimination stage.

Path (1): START → GAME OVER

- Video link: `https://www.bilibili.com/video/BV1JWB4BKEpe`

Path (2): START → CLEAR

(i) Original-speed video link: `https://www.bilibili.com/video/BV1VnBTBqERr`

(ii) 4×-speed video link: `https://www.bilibili.com/video/BV1VJBTBTEg2`

1.5 Discussion

1.5.1 Board Crossing and Visual Error Problem

In the second test (START → CLEAR), during the final stage of brick elimination, one may observe an abnormal phenomenon: although the board should collide with the ball, the ball appears to pass through the board. Moreover, the ball seems to be reflected by the bottom boundary.

However, according to the correctness of the overall gameplay (as shown in the full test video), the bottom wall should *not* reflect the ball, and the board should reflect the ball when they are in the correct relative positions. Therefore, the most plausible explanation is that this phenomenon is caused by VGA rendering and visual effects, rather than an actual logic error in the collision modules. Intuitively, the VGA display (and our frame update) is not perfectly smooth, which may lead to a visible mismatch between the rendered frame and the underlying collision timing. We attempted several approaches to mitigate this issue, including adjusting the ball velocity and changing the frame update speed. Unfortunately, these attempts did not fully eliminate the visual artifact. In future work, if we have further opportunities , we hope to revisit and address this problem again.

1.6 Conclusion

In this project, we designed and implemented an FPGA-based brick-breaker video game with VGA output, real-time user interaction, and a complete game-process controller. The system is organized in a modular manner: rendering modules generate pixel data for different pages/objects, while game-logic modules update the ball/paddle/bricks states at a frame-synchronous rate. To improve robustness of collision handling under discrete updates, we adopted a continuous-time (swept)

viewpoint inside each frame and converted it into deterministic state transitions, especially for the multiple-brick collision scenario. Moreover, we implemented a hierarchical FSM structure to manage the overall game process, including START, normal gameplay, life (hearts) management, and terminal states (CLEAR and GAME OVER).

Through virtual development board based simulation and recorded videos, we verified the correctness of the overall game process and the major functional requirements. We also observed a visual crossing/reflecting artifact in rare late-stage scenarios, which indicates that under certain relative positions and step sizes, the display-time sampling and the frame-time game update may produce a transient inconsistency. Future work may further improve consistency by refining the update granularity, applying swept collision logic to additional objects (e.g., paddle), or introducing sub-stepping when the ball velocity is large.

Overall, the project demonstrates a complete hardware-oriented workflow from algorithmic design (collision modeling and FSM logic) to practical verification, and provides a solid baseline for extending gameplay features such as power-ups, multiple balls, and richer UI effects.

.1 Full Verilog Code for Page Controllers

.1.1 screen_ctl.v

```
1  `timescale 1ns/1ps
2
3  //  FSMSTART -> MAIN_GAME -> (GAME_OVER / CLEAR) -> START
4  module screen_ctl (
5      input wire clk,
6      input wire rstn, // active-low asynchronous reset
7      input wire button, // raw button input, idle=1, pressed=0
8      input wire N_H_P,
9      input wire N_B,
10     output wire [2:0] frame,
11     output wire new_game_pulse //
12 );
13
14     // =============  =============
15     localparam [2:0] S_START = 3'd0,
16                      S_MAIN_GAME = 3'd1,
17                      S_GAME_OVER = 3'd2,
18                      S_CLEAR = 3'd3;
19
20     reg [2:0] st_cur, st_next;
21
22     // =============  =============
23     reg button_d;
24     wire btn_fall;
25
26     always @(posedge clk or negedge rstn) begin
27         if (!rstn)
28             button_d <= 1'b1;
29         else
30             button_d <= button;
31     end
32
33     // idle=1, pressed=01 -> 0
34     assign btn_fall = (button_d == 1'b1) && (button == 1'b0);
35
36     // =============  =============
```

```
37      always @(posedge clk or negedge rstn) begin
38          if (!rstn)
39              st_cur <= S_START;
40          else
41              st_cur <= st_next;
42      end
43
44      // =============  =============
45      always @(*) begin
46          st_next = st_cur; //
47
48          case (st_cur)
49              // START
50              S_START: begin
51                  if (btn_fall)
52                      st_next = S_MAIN_GAME;
53              end
54
55              //
56              S_MAIN_GAME: begin
57                  //  -> GAME_OVER
58                  if (N_H_P == 1'b1)
59                      st_next = S_GAME_OVER;
60                  //  -> CLEAR
61                  else if (N_B == 1'b1)
62                      st_next = S_CLEAR;
63                  //  MAIN_GAME
64              end
65
66              default: begin
67                  st_next = S_START;
68              end
69          endcase
70      end
71
72      // ============= Moore  =============
73      reg [2:0] frame_r;
74      always @(posedge clk or negedge rstn) begin
75          if (!rstn)
76              frame_r <= S_START;
```

```
77          else
78              frame_r <= st_cur;
79      end
80
81      assign frame = frame_r;
82
83      reg new_game_r;
84
85      always @(posedge clk or negedge rstn) begin
86          if (!rstn) begin
87              new_game_r <= 1'b0;
88          end else begin
89              new_game_r <= 1'b0; //  0
90
91              //  START
92              //
93              if (st_cur == S_START && btn_fall)
94                  new_game_r <= 1'b1;
95          end
96      end
97
98      assign new_game_pulse = new_game_r;
99
100 endmodule
```

.1.2 sub_FSM_ctl.v

```
1   `timescale 1ns/1ps
2
3   //  FSMCLEAR / GAME OVER
4   module sub_FSM_ctl (
5       input wire clk,
6       input wire rstn, // active-low asynchronous reset
7       input wire new_game, //  clk
8       input wire [2:0] frame, //  MAIN_GAME
9       input wire ball_alive, // 1: ; 0:
10      input wire no_bricks, // 1:
11
12      output wire N_H_P, //  -> GAME_OVER
```

```
13      output wire N_B, //  -> CLEAR
14      output wire reset_bb, //  ->  vga_pic_bbbw
15      output wire [1:0] life_level // 3,2,1,0
16  );
17      //  MAIN_GAME
18      localparam [2:0] FRAME_MAIN_GAME = 3'd1;
19
20      // 321
21      localparam [2:0] ST_3L = 3'd0;
22      localparam [2:0] ST_2L = 3'd1;
23      localparam [2:0] ST_1L = 3'd2;
24      localparam [2:0] ST_DIE = 3'd3;
25      localparam [2:0] ST_CLEAR = 3'd4;
26
27      reg [2:0] st_cur;
28      reg [1:0] life_r;
29      reg N_H_P_r;
30      reg N_B_r;
31      reg reset_bb_r;
32      reg ball_alive_d; //  ball_alive
33
34      wire in_game = (frame == FRAME_MAIN_GAME);
35      //   1  0
36      wire ball_lost_evt = in_game && (ball_alive_d == 1'b1) && (
            ball_alive == 1'b0);
37
38      always @(posedge clk or negedge rstn) begin
39          if (!rstn) begin
40              st_cur <= ST_3L;
41              life_r <= 2'd3;
42              N_H_P_r <= 1'b0;
43              N_B_r <= 1'b0;
44              reset_bb_r <= 1'b0;
45              ball_alive_d <= 1'b1; //
46          end else begin
47              //  ball_alive
48              ball_alive_d <= ball_alive;
49
50              //  reset_bb
51              reset_bb_r <= 1'b0;
```

```
52
53              // =3
54              if (new_game) begin
55                  st_cur <= ST_3L;
56                  life_r <= 2'd3;
57                  N_H_P_r <= 1'b0;
58                  N_B_r <= 1'b0;
59                  reset_bb_r <= 1'b1; // /
60              end else begin
61                  case (st_cur)
62                      // 3
63                      ST_3L: begin
64                          if (no_bricks) begin
65                              st_cur <= ST_CLEAR;
66                              N_B_r <= 1'b1;
67                          end else if (ball_lost_evt) begin
68                              st_cur <= ST_2L;
69                              life_r <= 2'd2;
70                              reset_bb_r <= 1'b1;
71                          end
72                      end
73
74                      // 2
75                      ST_2L: begin
76                          if (no_bricks) begin
77                              st_cur <= ST_CLEAR;
78                              N_B_r <= 1'b1;
79                          end else if (ball_lost_evt) begin
80                              st_cur <= ST_1L;
81                              life_r <= 2'd1;
82                              reset_bb_r <= 1'b1;
83                          end
84                      end
85
86                      // 1
87                      ST_1L: begin
88                          if (no_bricks) begin
89                              st_cur <= ST_CLEAR;
90                              N_B_r <= 1'b1;
91                          end else if (ball_lost_evt) begin
```

```
92                          st_cur <= ST_DIE;
93                          life_r <= 2'd0;
94                          N_H_P_r <= 1'b1; // GAME OVER
95                      end
96                  end
97
98                  //  GAME OVER N_H_P_r = 1 new_game
99                  ST_DIE: begin
100                     //
101                 end
102
103                 //  CLEAR N_B_r = 1 new_game
104                 ST_CLEAR: begin
105                     //
106                 end
107
108                 default: begin
109                     st_cur <= ST_3L;
110                     life_r <= 2'd3;
111                 end
112             endcase
113         end
114     end
115 end
116
117 assign life_level = life_r;
118 assign reset_bb = reset_bb_r;
119 assign N_H_P = N_H_P_r;
120 assign N_B = N_B_r;
121
122 endmodule
```

.1 Verilog Code for Overall Combination

.1.1 hearts_overlay.v

```
1 `timescale 1ns/1ps
2
3 module hearts_overlay (
```

```
4       input wire [9:0] pix_x,
5       input wire [9:0] pix_y,
6       input wire [1:0] life_level, // 3,2,1,0
7
8       output wire heart_pixel, //
9       output reg led1, // active-low
10      output reg led2,
11      output reg led3
12  );
13
14      //
15      localparam [9:0] H_VALID = 10'd640;
16      localparam [9:0] V_VALID = 10'd480;
17
18      //  vga_pic
19      localparam [9:0] HEART_SIZE = 10'd12;
20      localparam [9:0] HEART_SPACING = 10'd20;
21      localparam [9:0] HEART_START_X = H_VALID - 10'd350; //
22      localparam [9:0] HEART_START_Y = V_VALID - 10'd70;
23
24      //
25      wire in_heart1_area, in_heart2_area, in_heart3_area;
26      wire heart1_pixel, heart2_pixel, heart3_pixel;
27
28      assign in_heart1_area =
29          (pix_x >= HEART_START_X) &&
30          (pix_x < HEART_START_X + HEART_SIZE) &&
31          (pix_y >= HEART_START_Y) &&
32          (pix_y < HEART_START_Y + HEART_SIZE);
33
34      assign in_heart2_area =
35          (pix_x >= HEART_START_X + HEART_SPACING) &&
36          (pix_x < HEART_START_X + HEART_SPACING + HEART_SIZE) &&
37          (pix_y >= HEART_START_Y) &&
38          (pix_y < HEART_START_Y + HEART_SIZE);
39
40      assign in_heart3_area =
41          (pix_x >= HEART_START_X + 2*HEART_SPACING) &&
42          (pix_x < HEART_START_X + 2*HEART_SPACING + HEART_SIZE) &&
43          (pix_y >= HEART_START_Y) &&
```

```
44          (pix_y < HEART_START_Y + HEART_SIZE);
45
46      //
47      wire [9:0] heart1_lx = pix_x - HEART_START_X;
48      wire [9:0] heart1_ly = pix_y - HEART_START_Y;
49      wire [9:0] heart2_lx = pix_x - (HEART_START_X + HEART_SPACING);
50      wire [9:0] heart2_ly = pix_y - HEART_START_Y;
51      wire [9:0] heart3_lx = pix_x - (HEART_START_X + 2*HEART_SPACING);
52      wire [9:0] heart3_ly = pix_y - HEART_START_Y;
53
54      wire [9:0] heart1_half_size = HEART_SIZE >> 1;
55      wire [9:0] heart1_quarter_size = HEART_SIZE >> 2;
56      wire [9:0] heart2_half_size = HEART_SIZE >> 1;
57      wire [9:0] heart2_quarter_size = HEART_SIZE >> 2;
58      wire [9:0] heart3_half_size = HEART_SIZE >> 1;
59      wire [9:0] heart3_quarter_size = HEART_SIZE >> 2;
60
61      // 1
62      wire [19:0] heart1_left_circle =
63          (heart1_lx - heart1_quarter_size) * (heart1_lx -
                heart1_quarter_size) +
64          (heart1_ly - heart1_quarter_size) * (heart1_ly -
                heart1_quarter_size);
65      wire [19:0] heart1_right_circle =
66          (heart1_lx - 3*heart1_quarter_size) * (heart1_lx - 3*
                heart1_quarter_size) +
67          (heart1_ly - heart1_quarter_size) * (heart1_ly -
                heart1_quarter_size);
68
69      assign heart1_pixel = in_heart1_area && (
70          (heart1_left_circle <= heart1_quarter_size *
                heart1_quarter_size) ||
71          (heart1_right_circle <= heart1_quarter_size *
                heart1_quarter_size) ||
72          ((heart1_ly >= heart1_half_size) && (heart1_ly <= HEART_SIZE)
                &&
73           (heart1_lx >= (heart1_ly - heart1_half_size)) &&
74           (heart1_lx <= (HEART_SIZE - (heart1_ly - heart1_half_size))))
75      );
76
```

```
77      // 2
78      wire [19:0] heart2_left_circle =
79          (heart2_lx - heart2_quarter_size) * (heart2_lx -
                heart2_quarter_size) +
80          (heart2_ly - heart2_quarter_size) * (heart2_ly -
                heart2_quarter_size);
81      wire [19:0] heart2_right_circle =
82          (heart2_lx - 3*heart2_quarter_size) * (heart2_lx - 3*
                heart2_quarter_size) +
83          (heart2_ly - heart2_quarter_size) * (heart2_ly -
                heart2_quarter_size);
84
85      assign heart2_pixel = in_heart2_area && (
86          (heart2_left_circle <= heart2_quarter_size *
                heart2_quarter_size) ||
87          (heart2_right_circle <= heart2_quarter_size *
                heart2_quarter_size) ||
88          ((heart2_ly >= heart2_half_size) && (heart2_ly <= HEART_SIZE)
                &&
89           (heart2_lx >= (heart2_ly - heart2_half_size)) &&
90           (heart2_lx <= (HEART_SIZE - (heart2_ly - heart2_half_size))))
91      );
92
93      // 3
94      wire [19:0] heart3_left_circle =
95          (heart3_lx - heart3_quarter_size) * (heart3_lx -
                heart3_quarter_size) +
96          (heart3_ly - heart3_quarter_size) * (heart3_ly -
                heart3_quarter_size);
97      wire [19:0] heart3_right_circle =
98          (heart3_lx - 3*heart3_quarter_size) * (heart3_lx - 3*
                heart3_quarter_size) +
99          (heart3_ly - heart3_quarter_size) * (heart3_ly -
                heart3_quarter_size);
100
101     assign heart3_pixel = in_heart3_area && (
102         (heart3_left_circle <= heart3_quarter_size *
                heart3_quarter_size) ||
103         (heart3_right_circle <= heart3_quarter_size *
                heart3_quarter_size) ||
```

```
104         ((heart3_ly >= heart3_half_size) && (heart3_ly <= HEART_SIZE)
                &&
105          (heart3_lx >= (heart3_ly - heart3_half_size)) &&
106          (heart3_lx <= (HEART_SIZE - (heart3_ly - heart3_half_size))))
107     );
108
109     //  life_level
110     // life=3 -> 3life=2 -> 2life=1 -> 1life=0 ->
111     wire heart_enable_1 = (life_level >= 2'd1);
112     wire heart_enable_2 = (life_level >= 2'd2);
113     wire heart_enable_3 = (life_level >= 2'd3);
114
115     assign heart_pixel =
116          (heart_enable_1 && heart1_pixel)
117         || (heart_enable_2 && heart2_pixel)
118         || (heart_enable_3 && heart3_pixel);
119
120     // LED active-low
121     // life=3: led1=0, led2=0, led3=0
122     // life=2: led1=0, led2=0, led3=1
123     // life=1: led1=0, led2=1, led3=1
124     // life=0: led1=1, led2=1, led3=1
125     always @(*) begin
126         case (life_level)
127             2'd3: begin
128                 led1 = 1'b0;
129                 led2 = 1'b0;
130                 led3 = 1'b0;
131             end
132             2'd2: begin
133                 led1 = 1'b0;
134                 led2 = 1'b0;
135                 led3 = 1'b1;
136             end
137             2'd1: begin
138                 led1 = 1'b0;
139                 led2 = 1'b1;
140                 led3 = 1'b1;
141             end
142             default: begin // 0
```

```
143                 led1 = 1'b1;
144                 led2 = 1'b1;
145                 led3 = 1'b1;
146             end
147         endcase
148     end
149 
150 endmodule
```

.1.2 vga_scene.v

```
1  `timescale 1ns/1ps
2  
3  module vga_scene (
4      input wire vga_clk,
5      input wire sys_rst_n,
6      input wire [9:0] pix_x,
7      input wire [9:0] pix_y,
8      input wire [2:0] frame, // 0:START,1:GAME,2:OVER,3:CLEAR
9      input wire [1:0] life_level, //  sub_FSM_ctl
10     input wire [15:0] pix_data_game,//  vga_pic_bbbw
11 
12     output reg [15:0] pix_data,
13     output wire led1,
14     output wire led2,
15     output wire led3
16 );
17     //
18     localparam [15:0] RED = 16'hF800;
19 
20     //
21     wire [15:0] pix_start;
22     wire [15:0] pix_gameover;
23     wire [15:0] pix_clear;
24     wire heart_pixel;
25 
26     // START
27     vga_pic_start u_start (
28         .vga_clk (vga_clk),
```

```
29          .sys_rst_n (sys_rst_n),
30          .pix_x (pix_x),
31          .pix_y (pix_y),
32          .pix_data (pix_start)
33      );
34
35      // GAME OVER
36      vga_pic_gameover u_gameover (
37          .vga_clk (vga_clk),
38          .sys_rst_n (sys_rst_n),
39          .pix_x (pix_x),
40          .pix_y (pix_y),
41          .pix_data (pix_gameover)
42      );
43
44      // CLEAR
45      vga_pic_clear u_clear (
46          .vga_clk (vga_clk),
47          .sys_rst_n (sys_rst_n),
48          .pix_x (pix_x),
49          .pix_y (pix_y),
50          .pix_data (pix_clear)
51      );
52
53      //  + LED
54      hearts_overlay u_hearts (
55          .pix_x (pix_x),
56          .pix_y (pix_y),
57          .life_level (life_level),
58          .heart_pixel(heart_pixel),
59          .led1 (led1),
60          .led2 (led2),
61          .led3 (led3)
62      );
63
64      //  frame  MAIN_GAME
65      always @(*) begin
66          case (frame)
67              3'd0: begin
68                  // START
```

```
69                  pix_data = pix_start;
70              end
71
72              3'd1: begin
73                  //
74                  pix_data = pix_data_game;
75                  if (heart_pixel)
76                      pix_data = RED;
77              end
78
79              3'd2: begin
80                  // GAME OVER
81                  pix_data = pix_gameover;
82              end
83
84              3'd3: begin
85                  // CLEAR!
86                  pix_data = pix_clear;
87              end
88
89              default: begin
90                  pix_data = 16'h0000;
91              end
92          endcase
93      end
94
95  endmodule
```

.1.3 vga_colorbar.v

```
1   `timescale 1ns/1ps
2
3   module vga_colorbar(
4       input wire sys_clk,
5       input wire sys_rst_n,
6       input wire button_left,
7       input wire button_right,
8
9       output wire hsync,
```

```
10      output wire vsync,
11      output wire [15:0] rgb,
12
13      //   LED
14      output wire led1,
15      output wire led2,
16      output wire led3
17  );
18
19      //================  & VGA  ================
20      wire vga_clk ; // 25 MHz
21      wire [9:0] pix_x ; //  x
22      wire [9:0] pix_y ; //  y
23      wire [15:0] pix_data; //  vga_ctrl
24
25      pll pll_inst (
26          .sys_clk (sys_clk),
27          .sys_rst_n (sys_rst_n),
28          .vga_clk (vga_clk)
29      );
30
31      //================  FSM  ================
32      wire [2:0] frame; // 0=START,1=GAME,2=OVER,3=CLEAR
33      wire N_H_P; //
34      wire N_B; //
35      wire reset_bb; //  ->
36      wire [1:0] life_level; // 3/2/1/0
37
38      wire ball_alive_flag;
39      wire no_bricks_flag;
40
41      wire new_game_pulse; //
42
43      //================  FSM ==================
44      // /
45      screen_ctl u_screen (
46          .clk (vga_clk),
47          .rstn (sys_rst_n),
48          .button (button_left),
49
```

```
50          .N_H_P (N_H_P),
51          .N_B (N_B),
52          .frame (frame),
53          .new_game_pulse (new_game_pulse)
54      );
55
56      //=================  FSM//CLEAR =====
57      sub_FSM_ctl u_sub (
58          .clk (vga_clk),
59          .rstn (sys_rst_n),
60          .new_game (new_game_pulse), //   new_game_pulse
61          .frame (frame),
62          .ball_alive (ball_alive_flag),
63          .no_bricks (no_bricks_flag),
64
65          .N_H_P (N_H_P),
66          .N_B (N_B),
67          .reset_bb (reset_bb),
68          .life_level (life_level)
69      );
70
71      //=================  ======================
72      wire [9:0] x_board;
73
74      boardMoveCtl u_board_ctl (
75          .clk (vga_clk),
76          .sys_rst_n (sys_rst_n),
77          .new_game (new_game_pulse), //
78          .B1 (button_left),
79          .B2 (button_right),
80
81          .x_board (x_board)
82      );
83
84      //=================  +  +  =====
85      wire [15:0] pix_data_game;
86
87      vga_pic_bbbw u_bbbw (
88          .vga_clk (vga_clk),
89          .sys_rst_n (sys_rst_n),
```

```
90          .pix_x (pix_x),
91          .pix_y (pix_y),
92          .up (1'b0), //  0
93          .x_board (x_board),
94          .reset_bb (reset_bb), //  ->
95          .new_game (new_game_pulse),//  ->
96
97          .pix_data (pix_data_game),
98          .ball_alive_flag (ball_alive_flag),
99          .no_bricks_flag (no_bricks_flag)
100     );
101
102     //=================  ======================
103     //  START / GAME / GAMEOVER / CLEAR + /LED
104     vga_scene u_scene (
105         .vga_clk (vga_clk),
106         .sys_rst_n (sys_rst_n),
107         .pix_x (pix_x),
108         .pix_y (pix_y),
109         .frame (frame),
110         .life_level (life_level),
111         .pix_data_game (pix_data_game),
112
113         .pix_data (pix_data), //  VGA
114         .led1 (led1),
115         .led2 (led2),
116         .led3 (led3)
117     );
118
119     //================= VGA  ======================
120     vga_ctrl vga_ctrl_inst (
121         .vga_clk (vga_clk),
122         .sys_rst_n (sys_rst_n),
123         .pix_data (pix_data),
124
125         .pix_x (pix_x),
126         .pix_y (pix_y),
127         .hsync (hsync),
128         .vsync (vsync),
129         .rgb (rgb)
```

```
130     );
131     // ================= LED =================
132 // life_level = 0,1,2,3
133 // led1  1 led2  2 led3  3
134 
135 assign led1 = (life_level >= 2'd1) ? 1'b0 : 1'b1; //  1 LED1
136 assign led2 = (life_level >= 2'd2) ? 1'b0 : 1'b1; //  2 LED2
137 assign led3 = (life_level >= 2'd3) ? 1'b0 : 1'b1; //  3 LED3
138 
139 
140 endmodule
```